HyperVector-Optimizer: Correlation-Based Plan Routing for Filtered Vector Search, with a PostgreSQL Integration

Sanjay Mishra
Independent Researcher (IEEE Senior Member)
Raleigh, NC, USA
sanmish4@icloud.com

Divya Chukkapalli
Independent Researcher (IEEE Senior Member)
Raleigh, NC, USA
divya.95j@gmail.com

Abstract

Filtered top-k vector search—a nearest-neighbor query restricted by relational predicates and foreign-key semi-joins—can be executed by pre-filtering and scanning the survivors, by filtering inline during graph traversal, or by post-filtering an unfiltered graph search. Which plan is fastest depends less on the predicate's global selectivity than on its *local* selectivity around the query vector, and engines that route on global selectivity misroute exactly the correlated queries that matter. We present *HyperVector-Optimizer* (HVO), which routes each query on its local selectivity. (i) We define local selectivity operationally, as k divided by the number of tuples at least as close to the query as its k-th match: exactly the horizon a post-filtering plan must cover. (ii) The *Polytope-Ball Sketch* (HPBS) estimates it without touching the vector index. It partitions the vector space into k-means cells that carry radial distance profiles, per-attribute histograms and row-weighted foreign-key samples, and a 497-parameter calibrator corrects its residual bias. (iii) A minimax router prices the plans and picks the one whose worst-case cost over the estimator's uncertainty interval is smallest. The plan costs must be measured on the target machine rather than counted: costing every distance computation equally makes routing 1.6–6.3× slower. (iv) HVO runs as a PostgreSQL planner hook for `pgvector`. We prove a sensitivity lemma for doubling plans, regret bounds for point and minimax routing (the latter has zero regret whenever the exactly costed pre-filter plan is optimal), and a lower bound showing that, when predicates are evaluated through an index, any router using only global selectivity has regret that grows polynomially in the table size. We evaluate on two real datasets (one million arXiv papers and one million Amazon products, with real metadata and MiniLM embeddings) and two synthetic ones, running every plan for every query, against a resumable post-filter, ACORN, tuned global-selectivity routers and an HNSW-probing estimator. On arXiv, HPBS reaches a P99 local-selectivity Q-error of 6.17, against 39.23 for index probing and 765.57 for independence-based histograms, and it stays accurate on rare local matches (15.59 vs. 134.39). Inside PostgreSQL, stock `pgvector` and Exqutor (built from its released code) reach a mean Recall@10 of only 0.540 on arXiv. HVO is both faster and more accurate than `pgvector`'s forced iterative scan (16.10 vs. 18.04 ms; recall 0.953 vs. 0.907), and a recall-aware variant approaches the oracle's recall. Relative to the best tuned global-selectivity router, HVO lowers mean latency on arXiv by 43% (95% bootstrap interval 34–51%). On Amazon its 13% reduction is not distinguishable from zero (−4 to 28%), while the single-partition variant's 25% is (10–38%). P99 latency falls by up to 4×, and on synthetic data the means tie. HVO does not help at $k = 1$, where planning time dominates. We release the code, the patches, every experiment and both real benchmarks.

Keywords

Filtered vector search, approximate nearest neighbor search, selectivity estimation, query optimization, PostgreSQL

1 Introduction

Retrieval-augmented generation, recommendation, and analytical workloads increasingly issue *filtered* vector queries: a top-k similarity search over embeddings restricted by relational predicates and joins [1–4]. A typical query asks for the k papers most similar to a given one, but only among those that satisfy some conditions:

$$\begin{aligned}&\texttt{SELECT } r.\texttt{id FROM } R\ r \texttt{ JOIN } S\ s \texttt{ ON } r.\texttt{fk} = s.\texttt{pk}\\ &\texttt{WHERE } r.a_1 \in [l_1, u_1] \wedge r.a_2 \in C \wedge s.a_3 \in D\\ &\texttt{ORDER BY } \mathrm{dist}(r.\mathbf{v}, \mathbf{q}) \texttt{ LIMIT } k.\end{aligned} \tag{1}$$

General-purpose engines expose vector indexes inside the relational executor (PostgreSQL's `pgvector` [8], DuckDB's `vss` [9]), and specialized systems attach predicate bitmaps to graph and partition indexes (Milvus [10]). Whatever the engine, there are three ways to run such a query. *Pre-filtering* (Π_{pre}) evaluates the predicate first and then scans every qualifying vector, which is exact. *Post-filtering* (Π_{post}) walks an unfiltered HNSW graph [11] outward from the query and keeps candidates that pass the filter until it has k of them, as `pgvector`'s iterative scan and VBase [2] do. *Fused traversal* (Π_{fused}) filters inside the graph search itself [3, 10].

Which plan is fastest varies by orders of magnitude from query to query, and the information optimizers use today cannot tell. Figure 1 shows two real queries from our arXiv benchmark. Query A asks for the ten papers closest to *"Deep Active Learning for Dialogue Generation"* (a `cs.CL` paper), restricted to five physics archives. Query B asks for the ten papers closest to *"SIS Epidemic Spreading with Correlated Heterogeneous Infection Rates"* (`physics.soc-ph`), restricted to published papers in nine archives that include physics. Both filters match about the same fraction of the table: 5.9% and 6.6% of one million papers. A router that sees only this *global selectivity*—the information a histogram or an exact predicate count provides—must treat the two queries alike, and the best such router we could tune sends both to the pre-filter plan. That is right for Query A (4.03 ms) but 48× too slow for Query B, which post-filtering answers in 0.11 ms. A threshold between the two values would separate this particular pair, but a threshold applies to every query: among the 76 test queries whose global selectivity is within 15% of these two, local selectivity ranges from 0.031% to 56%, and the fastest correct plan is pre-filtering for 18% of them, post-filtering for 72% and fused traversal for 9%. Sending Query A to the post-filter would cost 17.25 ms and still miss some answers.

Intuitively, what separates the two queries is not how many papers match, but *where* they are. A dialogue-generation paper sits

in the computer-science region of the embedding space, far from physics papers; an epidemics paper filed under physics sits among them. A graph search starts at the query and moves outward, so it is cheap when matches are nearby and expensive when they are far away. *More concretely*, the post-filter must pass every tuple that is closer to the query than its k-th match. For Query B that is 34 papers; for Query A it is 13,490. We call k divided by this number the *local selectivity* of the filter around the query: 29.4% for Query B and 0.0741% for Query A, against global selectivities of about 6% for both. Positive correlation between the filter and the query's neighborhood makes local selectivity higher than global selectivity; anti-correlation makes it lower.

Recent work recognizes that global selectivity is not enough. Exqutor estimates vector-search cardinality inside the optimizer by probing the index or sampling [18], and benchmark studies document large performance swings under correlated filters [4, 6, 7]. Three questions remain open: what exactly the local selectivity that decides the plan is, how to estimate it cheaply without touching the index during planning, and how to route robustly, inside a real database, when the estimate is wrong.

In this paper we present *HyperVector-Optimizer* (HVO), which routes each query on its local selectivity. HVO defines local selectivity by the k-th match, as above, so that it measures exactly the distance a post-filter must travel. It estimates this quantity with the *Polytope-Ball Sketch* (HPBS), a compact synopsis that divides the vector space into regions and records, for each region, how far its members lie from its center and how often each attribute value occurs; a bisection then finds the ball around the query that holds k expected matches. HVO prices every plan with a cost model fitted to *measured* latencies on the target machine, and it chooses the plan whose cost is lowest in the worst case over the estimator's uncertainty. For Query A, HPBS estimates a local selectivity of 0.0578% (true value 0.0741%) and HVO pre-filters; for Query B it estimates 12.4% (true 29.4%) and HVO post-filters. Both choices are the fastest plans that return the correct answer. We prove that this minimax rule never misroutes a query whose best plan is the exact pre-filter scan, provided the uncertainty interval covers the truth, and that any router that sees only global selectivity can be arbitrarily bad as the table grows. HVO also runs inside PostgreSQL, as a planner hook for `pgvector`.

We evaluate HVO on two real datasets (one million arXiv papers and one million Amazon products, with real metadata and MiniLM embeddings) and two synthetic ones, executing every plan for every test query so that the best plan is measured rather than modeled. The example of Figure 1 is not rare: on the arXiv test queries, HVO and the best global router disagree on 15% of queries, and HVO's choice is faster on 89% of those. Overall, HVO lowers mean latency on arXiv by 43% (95% bootstrap interval 34–51%) and cuts tail latency up to fourfold. On Amazon its tail gain is clear but its mean gain is not distinguishable from zero, and on synthetic data the means tie. Inside PostgreSQL, stock `pgvector` and Exqutor (built from its released code) return on average only about 0.540 of the true top-10 for these queries, and HVO is both faster and more accurate than `pgvector`'s iterative scan. HVO does not help at $k = 1$, where planning time dominates. We release the code, every experiment, and both real benchmarks with exact ground truth [32].

2 Background and Related Work

Hybrid execution engines. Relational engines with vector support include AnalyticDB-V [1], VBase [2], which iterates HNSW results under relaxed monotonicity until predicates are satisfied, `pgvector` with iterative index scans [8], and DuckDB `vss` [9]. Milvus [10] evaluates scalar predicates into a bitset that the vector kernel consults during search. *Limitation:* these systems choose among plans with rules or with cost models driven by global predicate selectivity, which cannot tell a predicate whose matches surround the query from one whose matches lie far away.

Filtered ANN indexes. A second line of work bakes attributes into the index. Filtered-DiskANN [13] and NHQ [14] incorporate labels into graph construction; HQANN [15] fuses attribute and vector distances; SeRF [16] and its dynamic successor [17] build segment graphs for one-dimensional range filters; ACORN [3] densifies HNSW and expands two-hop neighborhoods to stay connected under arbitrary predicates. Surveys and benchmarks [4–7] find that no index dominates across selectivities. *Limitation:* each index fixes one filtering strategy. None decides, per query, whether another available plan would be cheaper, so whichever indexes exist, the optimizer still has to choose.

Cardinality estimation. Relational estimators range from histograms and wavelets [20, 21] through sketches such as HyperLogLog and k-minimum values [22, 23] to learned models [24–26]. For similarity queries, estimators use LSH [27, 28] or reference objects [29]. Exqutor [18] is closest to us: it plugs vector-search cardinality estimates into a relational optimizer, using index-based techniques when an HNSW or IVF index exists and adaptive sampling otherwise. *Limitation:* relational estimators assume that predicates are independent of the vector; similarity estimators count matches within a fixed range rather than the k-th-match horizon that determines graph-plan cost; and index probing must touch the index during planning and can only bound the selectivity of rare matches.

Synthesis. Table 1 summarizes the differences. HVO differs from the closest work in three ways. It estimates the *joint* quantity (matches inside a ball) from a synopsis without probing the index; it targets the k-th-match horizon that determines graph-plan cost; and it pairs the estimate with measured plan costs, regret analysis and a PostgreSQL integration. We compare against an Exqutor-style index probe and a sampling estimator inside our benchmark, and against Exqutor itself, built from its released code, inside PostgreSQL.

3 Filtered Vector Search, by Example

This section explains the problem on the running example of Figure 1 before we formalize it. We first fix the setting (Section 3.1), then follow the two queries through the three plans (Section 3.2), and finally show why global selectivity cannot predict the best plan (Section 3.3).

3.1 Setting and Assumptions

A fact table R holds N tuples, each with a unit-norm vector $\mathbf{v}_i \in \mathbb{R}^d$ and relational attributes $\mathbf{a}_i$; a foreign key may reference a small dimension table. A query $(\mathbf{q}, k, \mathcal{P})$ asks for the k tuples with the largest inner product with $\mathbf{q}$ among those that satisfy the predicate

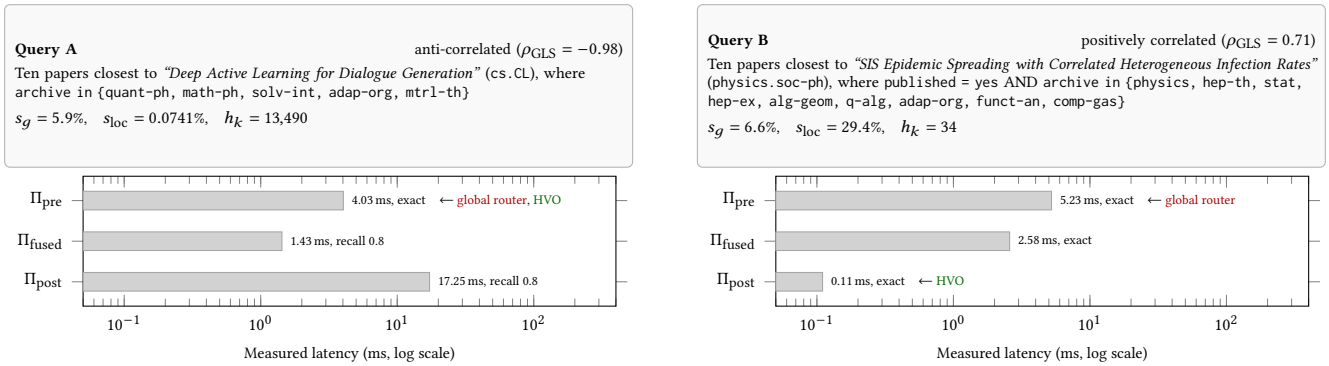


Figure 1: A real example from the ARXIV-384 benchmark ($k = 10$). The two filters match about the same share of the table, so a router that sees only global selectivity sends both queries to the same plan (here pre-filtering, below its tuned threshold of 7.9%). Query B's matches surround the query paper and post-filtering answers it 48× faster; Query A's matches lie far away and post-filtering must pass 13,490 papers. HVO estimates the local selectivity and routes both queries to their fastest plan that returns the correct answer. "recall": Recall@10 of the approximate graph plans. The pair was selected from the test split by a fixed rule to illustrate the failure mode (Section 3); Section 5 reports how often it occurs.

Table 1: How existing approaches choose a plan for filtered top-k vector search. "Index at planning": whether choosing the plan requires accessing the vector index.

Approach	Plan choice	Estimate driving the choice	Index at planning	Implemented in
AnalyticDB-V [1]	cost-based, among hybrid plans	global predicate selectivity	no	own analytical engine
VBase [2]	one iterative plan	–	–	PostgreSQL-based engine
pgvector iterative scan [8]	PostgreSQL cost model	planner statistics (global)	no	PostgreSQL extension
Milvus [10]	bitset-filtered search	–	–	vector database
Filtered indexes [3, 13, 14, 16]	none (one index)	–	–	libraries
Exqutor [18]	cost-based, in the optimizer	vector-search cardinality (probe or sampling)	yes, when an index exists	PostgreSQL
HVO (this work)	minimax over an uncertainty interval	k-th-match local selectivity (sketch)	no	benchmark and PostgreSQL hook

$\mathcal{P}$, a conjunction of range, IN and semi-join terms as in Eq. (1). In ARXIV-384, R holds one million papers with 384-dimensional MiniLM embeddings; the attributes include the paper's archive, publication status, year and author count, and the foreign key references its primary category. Table 2 lists the notation.

We assume that every plan adapts its effort until it has k answers (or has exhausted its budget), so a plan's latency does not depend on what the optimizer believed; the optimizer only chooses *which* plan runs. This makes the best plan of every query measurable: we run all three and take the fastest whose Recall@k is at least 0.9 (the *oracle*). Plans run single-threaded. Everything the optimizer learns—cost constants, the calibrator, the uncertainty interval—is fitted on training queries and evaluated on disjoint test queries.

3.2 Three Plans, Two Queries

Table 3 follows Query A and Query B through the three plans. The plans are described in full in Section 4.3; here we only need what each one does.

Pre-filtering evaluates the filter over all one million rows, which costs a few milliseconds regardless of the query, and then scans the s_gN matching vectors: 58,544 for Query A and 66,356 for Query B. Its cost depends only on the global selectivity, and its answer is exact. It takes 4.03 and 5.23 ms.

Post-filtering starts at the query's position in the HNSW graph and returns tuples in roughly increasing distance, checking each against the filter, until k have passed. For Query B the tenth match is the 34-th closest paper, so the scan stops almost immediately (0.11 ms). For Query A it is the 13,490-th closest: the scan must pass more than ten thousand computer-science papers before it reaches enough physics papers, and it takes 17.25 ms. Because it walks a graph rather than scanning, it can also miss true answers (Recall@10 0.8).

Fused traversal (ACORN-1 [3]) filters inside the graph search. It needs the filter evaluated first, like pre-filtering, but then explores only matching nodes. It is fast on both queries here (1.43 and 2.58 ms), but on Query A it returns only 0.8 of the true top-10.

The fastest plan that returns a correct answer is therefore pre-filtering for Query A and post-filtering for Query B, and the difference is large: choosing wrongly costs 4× on Query A and 48× on Query B.

3.3 Understanding Why Global Selectivity Fails

The example suggests that the plan is decided by how far the graph search must travel before it meets k matches. We now make that precise and show, first on the example and then across the workload, that global selectivity does not carry this information.

Table 2: Notation, acronyms and units.

Data and queries	
R, N, d	fact table, its number of tuples, vector dimension
$\mathbf{v}_i$, $\mathbf{a}_i$	unit-norm vector and attributes of tuple i
$(\mathbf{q}, k, \mathcal{P})$	query vector, number of results, predicate (conjunction of range, IN and semi-join terms)
$V(\mathcal{P})$	set of tuples that match $\mathcal{P}$
s_g	global selectivity, $\lvert V(\mathcal{P})\rvert/N$
h_k	tuples at least as close to $\mathbf{q}$ as its k-th match (Definition 3.1)
s_{loc}	k-th-match local selectivity, k/h_k
ρ_{GLS}	global-local selectivity coefficient, Eq. (2)
Q-error	$\max(\hat{c}/c,\ c/\hat{c})$ for an estimate $\hat{c}$ of c
Sketch (HPBS)	
L, m, B	partitions, cells per partition, histogram bins per attribute
$\mathbf{c}_b$, N_b	centroid and size of cell b
$\rho_{b,j}$	j-th of 16 quantiles of squared member-to-centroid distances
δ_b	squared distance $\lVert\mathbf{q}-\mathbf{c}_b\rVert^2$
$\hat{n}_b(r)$, $\hat{\sigma}_b$	estimated tuples of cell b in the ball of squared radius r; estimated matching fraction of cell b
k_{kmv}, $\mathcal{K}_b$, $\mathcal{V}$	bottom-k size, key sample of cell b, valid dimension keys
f_Θ	residual calibrator (MLP)
Plans, costs and routing	
Π_{pre}, Π_{fused}, Π_{post}	pre-filter, fused (ACORN-1) and post-filter plans
$C(\Pi)$, C_{rel}	plan cost; cost of columnar predicate evaluation
c_{scan}, a_0	cost per scanned vector; fixed pre-filter cost
$\text{St}(\nu; b_0, b_1, \gamma)$	staircase cost of a doubling plan with need ν, Eq. (6)
c_f, β_f	need multiplier and exponent of the fused plan
η	width of the uncertainty interval $[\hat{s}/\eta, \hat{s}\eta]$
Λ	cost sensitivity factor (Lemma 4.2)
$\mathcal{R}$	regret, $C(\hat{\Pi})/C(\Pi^*)$
s_g^*	pre-filter boundary (Theorem 3.2)
Acronyms and units	
HVO, HPBS	HyperVector-Optimizer; Polytope-Ball Sketch
ANN, HNSW	approximate nearest neighbor; hierarchical navigable small world graph
AVI, KMV, FK	attribute-value independence; k-minimum values; foreign key
CM, CI	cost model; confidence interval (95%, percentile bootstrap)
ms, μs, MiB	milliseconds, microseconds, mebibytes

Table 3: The running example in numbers ($k = 10$). Latency in ms (Recall@10 in parentheses when below 1). "Global": the plan chosen by the best tuned global-selectivity router; "HVO": the plan HVO chooses; "Oracle": the fastest plan with Recall@10 ≥ 0.9.

	Query A (anti)	Query B (positive)
Global selectivity s_g	5.9%	6.6%
h_k (tuples before 10th match)	13,490	34
Local selectivity k/h_k	0.0741%	29.4%
Π_{pre} (pre-filter)	4.03	5.23
Π_{fused} (ACORN-1)	1.43 (0.8)	2.58
Π_{post} (post-filter)	17.25 (0.8)	0.11
Oracle / Global / HVO	Π_{pre} / Π_{pre} / Π_{pre}	Π_{post} / Π_{pre} / Π_{post}

The k-th-match horizon. The post-filter's work is governed by h_k, the number of tuples it must pass. We define local selectivity directly from it:

Definition 3.1 (k-th-match local selectivity). Order R by distance to $\mathbf{q}$. Let $h_k(\mathbf{q}, \mathcal{P})$ be the number of tuples whose distance to $\mathbf{q}$ is at most that of the k-th closest tuple in $V(\mathcal{P})$. The local selectivity is $s_{\text{loc}}(\mathbf{q}, \mathcal{P}) = k/h_k(\mathbf{q}, \mathcal{P})$.

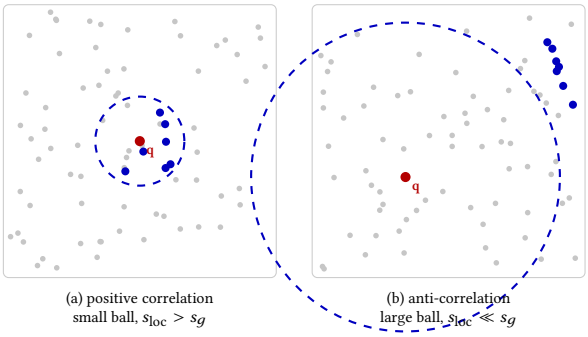


Figure 2: Schematic of local selectivity (illustration, not data). Gray: tuples that fail the filter; blue: matches; red: the query. The dashed ball is the smallest ball around q that contains k matches, and h_k is the number of tuples inside it. Both panels have the same global selectivity. A post-filter must pass every tuple in the ball, so its cost follows $h_k = k/s_{\text{loc}}$, not s_g.

Equivalently, s_{loc} is the fraction of matches inside the smallest ball around $\mathbf{q}$ that contains k matches. Under independence between $\mathcal{P}$ and the vectors, $h_k \approx k/s_g$ and $s_{\text{loc}} \approx s_g$. An earlier version of this work measured the match fraction inside a ball of fixed size k/s_g. That quantity predicted graph-plan cost poorly under anti-correlation: there the k-th match lies far outside that ball, so the plan must search much further than the fixed ball suggests. Definition 3.1 measures that distance directly. We quantify correlation by the global-local selectivity coefficient [4]

$$\rho_{\text{GLS}} = \tanh\left(\tfrac{1}{2}\ln\frac{s_{\text{loc}}(1-s_g)}{s_g(1-s_{\text{loc}})}\right) \in (-1, 1), \tag{2}$$

and call a query anti-correlated if $\rho_{\text{GLS}} < -0.3$, positively correlated if $\rho_{\text{GLS}} > 0.3$, and independent otherwise.

The optimizer must choose $\hat{\Pi} \in \{\Pi_{\text{pre}}, \Pi_{\text{fused}}, \Pi_{\text{post}}\}$ from estimates $(\hat{s}_g, \hat{s}_{\text{loc}})$ produced in planning time that is small relative to execution.

Figure 2 illustrates the definition. When the filter is positively correlated with the query's neighborhood (left), the k matches lie close to $\mathbf{q}$, the smallest ball containing them is small, and $s_{\text{loc}} > s_g$. When it is anti-correlated (right), the ball must grow until it reaches the region where matches live, and $s_{\text{loc}} \ll s_g$. In the example the two queries have nearly the same s_g (about 6%) but local selectivities that differ by a factor of four hundred.

It is not just the example. Among the 76 ARXIV-384 test queries whose global selectivity lies within 15% of the example's, local selectivity spans 0.031% to 56%, and the fastest correct plan is pre-filtering for 18%, post-filtering for 72% and fused traversal for 9% of them. Any rule that maps s_g to a plan must route all of them the same way.

Where each plan wins. Once plan costs are written as functions of s_g and s_{loc} (Section 4.4), the region where pre-filtering is cheapest has a simple shape.

Theorem 3.2 (Pre-filter boundary). *Let $C_G(s_{loc})$ denote the cost of the cheaper graph plan,* $\min(C(\Pi_{fused}), C(\Pi_{post}))$. *Under the*

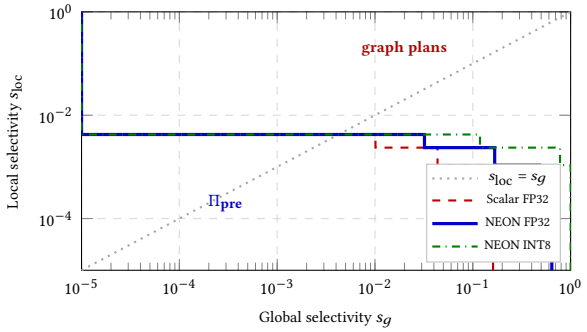


Figure 3: Pre-filter boundary $s_g^*(s_{\text{loc}})$ of Theorem 3.2 on SYN-128 for the three scan kernels (cost model calibrated on the training split; two predicate terms, no join). Below and to the left of each curve pre-filtering is cheapest. Faster kernels push the boundary to larger s_g. Above $s_{\text{loc}} \approx 1\%$ a graph plan wins for every s_g, because columnar predicate evaluation already costs $\Theta(N)$.

cost model of Eqs. (7) *to* (9)*, the pre-filter plan is optimal if and only if*

$$s_g \le s_g^*(s_{loc}) = \frac{C_G(s_{loc}) - C_{rel} - a_0}{c_{scan} N}.$$

If $C_G(s_{loc}) < C_{rel} + a_0$, Π_{pre} is never optimal. Holding the other terms fixed, replacing a scan kernel by one that is λ times faster multiplies s_g^* by λ.

Proof. $C(\Pi_{\text{pre}})$ is affine and increasing in s_g, and C_G does not depend on s_g; equate and solve. The scaling follows because c_{scan} enters only in the denominator. □

Theorem 3.2 has a consequence that matters in practice. When predicates are evaluated by scanning columns, C_{rel} is linear in N, so pre-filtering pays $\Theta(N)$ even for very selective predicates. For local selectivities above roughly 1–2%, a graph plan is then cheaper whatever s_g is. The boundary becomes nearly a horizontal line in the (s_g, s_{loc}) plane, and global selectivity carries little information about the right plan (Figure 3). The kernel enters through c_{scan}, which on SYN-128 is 64.4, 16.0, and 3.3 ns per vector for the scalar, NEON FP32 and INT8 kernels.

4 HyperVector-Optimizer

We now describe how HVO makes the decision of Figure 1 automatically, in planning time that is small compared with execution.

4.1 Overview

Intuitively, HVO needs to answer one question before a query runs: roughly how many tuples lie between the query and its k-th match? If it knew that number, it could price each plan and pick the cheapest. HVO answers it from a precomputed summary of the data, prices the plans with costs measured on the machine, and hedges against its own estimation error.

Figure 4 shows the pipeline, which runs in four steps for every query. (1) The predicate encoder turns $\mathcal{P}$ into histogram positions, `IN` masks and the set of valid dimension keys. (2) The sketch, HPBS (Section 4.2), combines the encoded predicate with the query vector and returns estimates $\hat{s}_g$ and $\hat{s}_{\text{loc}}$, which a small calibrator corrects. (3) The router (Section 4.5) evaluates a cost model fitted to measured latencies (Section 4.4) at both ends of an uncertainty interval around the estimate, optionally excludes graph plans whose predicted recall is too low, and picks the plan with the smallest worst-case cost. (4) The chosen plan runs. Planning reads only the sketch, never the vector index. Everything fitted is fitted offline on training queries, and the same native planner serves our benchmark and the PostgreSQL extension (Section 4.7).

4.2 The Polytope-Ball Sketch

HPBS answers the question *"how many tuples lie in the ball around* $\mathbf{q}$ *that holds k matches of $\mathcal{P}$?"* The ball is a region in $\mathbb{R}^d$; the predicate is an axis-aligned polytope in attribute space joined through foreign keys. Space-partitioning trees degrade to scans for $d \gtrsim 30$, and the predicate is not known until query time, so we summarize both sides per cell of a coarse vector partition.

Intuitively, HPBS keeps a coarse map of the vector space. It divides the space into regions (cells) and remembers, for every cell, how far its members lie from the cell's center and how its attribute values are distributed. Given a query, it can then ask each cell: "how many of your members are within distance r of the query, and what fraction of them pass the filter?" Summing the answers over cells gives the expected number of matches within distance r, and a bisection on r finds the ball that holds k of them. The number of tuples in that ball estimates h_k.

4.2.1 Structure. Partitions. We build L independent k-means partitions with m cells each, trained on a 2×10^5-vector sample with different seeds. For cell b we keep its centroid $\mathbf{c}_b$, its size N_b, and a *radial profile*: the 16 quantiles $\{\rho_{b,j}\}$ of the squared member-to-centroid distances $\|\mathbf{x} - \mathbf{c}_b\|^2$.

Attribute histograms. For each numeric attribute we fix $B = 32$ global equi-depth bins and store, per cell, the cumulative fraction of the cell's tuples below each bin edge (FP16). For integer-valued attributes (years, counts) the edges are moved to half-integers and an inclusive range $[l, u]$ is read as $[\lceil l \rceil - \frac{1}{2}, \lfloor u \rfloor + \frac{1}{2}]$. Without this, tied values make equi-depth edges collapse; a first run on real data produced P99 Q-errors above 10^4 for exactly this reason. For each categorical attribute we store per-cell value fractions. A range predicate is answered by linear interpolation inside its two boundary bins; an `IN` list sums the selected fractions.

Foreign-key samples. For each foreign key, a cell stores its $k_{\text{kmv}} = 64$ distinct keys with the smallest hash values together with the number of the cell's rows carrying each key. At query time the dimension predicate is evaluated on the (small) dimension table to obtain the valid key set $\mathcal{V}$. The cell's semi-join selectivity is estimated as

$$\hat{\sigma}_b^{\text{join}} = \frac{\sum_{h \in \mathcal{K}_b} f_b(h)\, \mathbb{I}[h \in \mathcal{V}]}{\sum_{h \in \mathcal{K}_b} f_b(h)}, \tag{3}$$

a ratio estimator over a uniform hash-sample of the cell's distinct keys that weights each key by its row count. Because $\mathcal{V}$ is materialized exactly, no threshold alignment between two bottom-k sketches is needed. The estimator is consistent but not unbiased, and its variance grows with key skew.

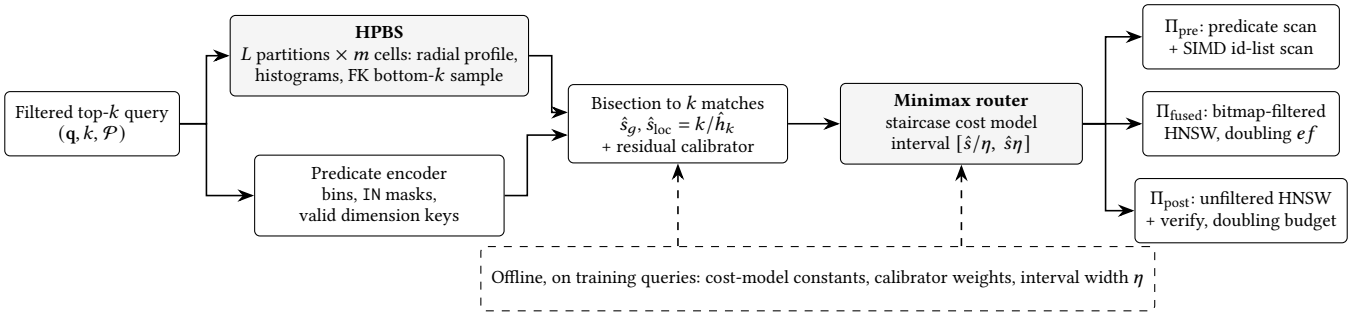


Figure 4: Architecture of HVO. Planning reads only the sketch, never the vector index. The router evaluates a calibrated cost model at both ends of the estimator's uncertainty interval and runs the plan with the smallest worst-case cost.

Table 4: Algorithm 1: HPBS query-time estimation (one partition; the sketch returns the median over L).

Input: $\mathbf{q}$, k, predicate $\mathcal{P}$, partition $\{\mathbf{c}_b, N_b, \rho_{b,\cdot}, \text{hist}_b, \mathcal{K}_b\}_{b=1}^{m}$.
1: Encode $\mathcal{P}$: boundary bins and offsets of each range, bitmask of each IN list, valid key set $\mathcal{V}$ of the dimension predicate.
2: $\boldsymbol{\delta} \leftarrow \|\mathbf{c}\|^2 - 2\,\mathbf{C}\mathbf{q} + 1$ ▷ one sgemv
3: **for** each cell b: $\hat{\sigma}_b \leftarrow \prod_j \hat{\sigma}_b^{(j)} \cdot \hat{\sigma}_b^{\text{join}}$ (Eq. (3)); $\hat{s}_g$ += $N_b\hat{\sigma}_b/N$.
4: $C \leftarrow \{b : \hat{\sigma}_b > 0\}$; $lo \leftarrow 0$; $hi \leftarrow \max_{b\in C} \delta_b + \rho_{b,16}$.
5: **repeat** 26 times: $r \leftarrow (lo + hi)/2$; **if** $\sum_{b\in C} \hat{n}_b(r)\hat{\sigma}_b < k$ **then** $lo \leftarrow r$ **else** $hi \leftarrow r$; $C \leftarrow \{b \in C : \delta_b \le hi\}$.
6: **return** $\hat{s}_g$, $\hat{s}_{\text{loc}} \leftarrow \min(1, k/\sum_b \hat{n}_b(hi))$.

Cell predicate fraction. Within a cell we assume the constraints are independent: $\hat{\sigma}_b = \prod_j \hat{\sigma}_b^{(j)} \cdot \hat{\sigma}_b^{\text{join}}$. Correlation *across* cells, which is the spatial-relational correlation that defeats global estimators, is captured by the partition; only the within-cell residual is assumed away.

Why k-means and not random projections. Our original design used structured cross-polytope LSH cells ($2d_{\text{proj}}$ cones × 8 radial shells). Its cells follow random directions rather than the data, so a cell mixes unrelated neighborhoods. In our experiments it is the least accurate variant (P99 Q-error 31.59 against 9.83 for k-means cells at $L = 4$; Table 7).

4.2.2 Estimation. Let $\delta_b = \|\mathbf{q} - \mathbf{c}_b\|^2$. For high-dimensional data the offset of a member from its centroid is nearly orthogonal to $\mathbf{q} - \mathbf{c}_b$, so $\|\mathbf{q} - \mathbf{x}\|^2 \approx \delta_b + \|\mathbf{x} - \mathbf{c}_b\|^2$. The estimated number of the cell's tuples inside the ball of squared radius r is therefore

$$\hat{n}_b(r) = \frac{N_b}{16}\left|\{j : \delta_b + \rho_{b,j} \le r\}\right|. \tag{4}$$

The estimated matches inside the ball are $\hat{M}(r) = \sum_b \hat{n}_b(r)\hat{\sigma}_b$. We find $\hat{r}$ with $\hat{M}(\hat{r}) = k$ by bisection (26 steps), and return

$$\hat{s}_{\text{loc}}^{(\ell)} = \min\Big(1,\ \frac{k}{\sum_b \hat{n}_b(\hat{r})}\Big), \qquad \hat{s}_g^{(\ell)} = \frac{1}{N}\sum_b N_b\hat{\sigma}_b. \tag{5}$$

The sketch returns the median over the L partitions. Algorithm 1 gives the procedure. Each partition costs one $m \times d$ matrix-vector product (Accelerate sgemv), m cell-fraction evaluations, and a bisection whose candidate set shrinks as the upper radius falls, giving $O(L(md + m|\mathcal{P}| + m\log(1/\epsilon)))$ time.

4.2.3 Residual calibrator. The raw estimate is biased where the orthogonality approximation in Eq. (4) is poor (the query lies close to a centroid) and where within-cell independence fails. A two-layer MLP $f_\Theta : \mathbb{R}^{12} \to \mathbb{R}$ with 16 ReLU units per layer (497 parameters) predicts the log-residual, and $\hat{s}_{\text{loc}} \leftarrow \text{clip}(\hat{s}_{\text{loc}}^{\text{raw}} e^{f_\Theta(\mathbf{z})}, k/N, 1)$. The twelve features are: $\ln \hat{s}_g$; $\ln \hat{s}_{\text{loc}}^{\text{raw}}$; the variance and the range of $\ln \hat{s}_{\text{loc}}^{(\ell)}$ across partitions; the number of predicate terms; a join indicator; the number of range terms; the number of IN terms; the log of the mean number of active cells; the mean and the standard deviation across partitions of the bisection radius $\hat{r}$; and $\ln \hat{s}_{\text{loc}}^{\text{raw}} - \ln \hat{s}_g$. It is trained on the training split only, with squared log-error, L_2 penalty 10^{-3} and early stopping. Its forward pass takes about 4.0 μs in NumPy.

Example. For Query A, the filter selects five physics archives. Cells in the computer-science region of the space hold almost no physics papers, so the estimated ball must grow until it reaches cells where physics papers are common. The raw sketch estimates $\hat{s}_{\text{loc}} = 0.0588\%$ and the calibrated estimate is 0.0578%, against a true 0.0741%. An HNSW probe of the query's 256 nearest neighbors finds no match at all and can only report its floor of 0.195% (Section 5.2). For Query B, nearby cells contain many published physics papers; the raw estimate is 6.23%, the calibrated one 12.4%, and the truth 29.4%. Both estimates are within a factor of 2.5 of the truth, which is all the router needs here.

4.2.4 Error decomposition. Let $n_b(r)$ and $M_b(r)$ be the true numbers of tuples and of matches of cell b inside the ball of squared radius r, and $\sigma_b(r) = M_b(r)/n_b(r)$. Write $h(j)$ for the number of tuples within the distance of the j-th closest match, so $h_k = h(k)$.

Proposition 4.1 (Deterministic error decomposition). *Suppose that for every r and b the geometric and cell estimates satisfy*

$$\hat{n}_b(r) \in \big[n_b(r)/\alpha,\ \alpha n_b(r)\big], \qquad \hat{\sigma}_b \in \big[\sigma_b(r)/\beta,\ \beta\sigma_b(r)\big].$$

Then

$$\textit{Q-error}(\hat{s}_{loc}, s_{loc}) \le \alpha \cdot \psi(\alpha\beta), \quad \psi(t) = \max\Big(\frac{h(\lceil tk\rceil)}{h(k)},\ \frac{h(k)}{h(\lfloor k/t\rfloor)}\Big).$$

If matches are locally uniform around $\mathbf{q}$ *($h(j) \propto j$), then $\psi(t) \approx t$ and the Q-error is at most $\alpha^2\beta$.*

Proof. With $M(r) = \sum_b n_b(r)\sigma_b(r)$,

$$\hat{M}(r) = \textstyle\sum_b \hat{n}_b\hat{\sigma}_b \in \big[M(r)/(\alpha\beta),\ \alpha\beta M(r)\big].$$

At the bisection point $\hat{M}(\hat{r}) = k$, so $M(\hat{r})$ lies in $[k/(\alpha\beta),\ \alpha\beta k]$. Since M and $n = \sum_b n_b$ are non-decreasing in r,

$$n(\hat{r}) \in \big[h(\lfloor k/(\alpha\beta)\rfloor),\ h(\lceil \alpha\beta k\rceil)\big].$$

Finally $\sum_b \hat{n}_b(\hat{r}) \in [n(\hat{r})/\alpha,\ \alpha n(\hat{r})]$, and $\hat{s}_{\text{loc}}/s_{\text{loc}} = h(k)/\sum_b \hat{n}_b(\hat{r})$. □

What the parameters control. Smaller cells (larger m) and a finer radial profile reduce α; larger B and k_{kmv} reduce the histogram and join parts of β. The remaining part of β is within-cell spatial-relational correlation, and ψ depends on how matches are arranged around $\mathbf{q}$. Neither is controlled by the sketch's parameters. We therefore make no distribution-free accuracy claim: Proposition 4.1 explains where error comes from, and Section 5.2 measures it. We prove nothing about the calibrator.

Space and maintenance. HPBS stores $Lm(d + 16)$ floats for centroids and profiles, $Lm\sum_j B_j$ FP16 histogram entries, and Lmk_{kmv} key–count pairs, independent of N apart from the choice of m. With $L = 4$, $m = 1024$ it takes 5.1, 9.1 and 15.1 MiB for 128-, 384- and 768-d vectors (1.05%, 0.62% and 0.52% of the raw vectors), dominated by the centroids. It is built in 10–24 s, against 93–214 s for the HNSW index on 10 threads. Insertions update one cell per partition (counts, histogram entries, and a bottom-k insertion). Deletions can be handled by decrementing counts and dropping key entries, with lazy cell rebuilds. We have not evaluated maintenance under updates.

4.3 Plans

Graph plans run on a faiss [12] `IndexHNSWFlat` graph (inner product, $M = 32$, $ef_{\text{construction}} = 200$) or on ACORN's own graph; every plan runs single-threaded.

- Π_{pre} evaluates $\mathcal{P}$ column-wise over all N rows (NumPy), takes the sorted id list, and runs a C top-k scan over the listed rows. The scan prefetches whole rows ahead and comes in three kernels: scalar FP32 (auto-vectorization disabled), NEON FP32 (4×4 FMA accumulators), and NEON INT8 (`SDOT` on symmetric int8 codes for the top $4k$, followed by an exact FP32 rerank).
- Π_{post} is a *resumable* iterative scan over the HNSW graph, in the spirit of `pgvector`'s iterative index scan. It keeps the best $ef = \max(64, k)$ visited but not yet returned nodes in a window and expands candidates while the nearest unexpanded candidate is closer than the window's worst node, which is HNSW's stopping rule. It then returns the window's closest node and refills the window from an overflow heap. Batches of $k, k, 2k, 4k, \ldots$ nodes are checked against $\mathcal{P}$ until k matches are found (budget 2^{17}), and each batch resumes the same search state, so no work is repeated.
- Π_{fused} is ACORN-1 [3] (reference implementation, $M = 32$, $M_\beta = 64$, $\gamma = 1$). The predicate is materialized as a byte map and ACORN expands filtered two-hop neighborhoods during traversal. ACORN doubles ef from 64 until k results are found *and* a doubled budget returns the same set. We report ACORN-1 rather than ACORN-γ: with $\gamma = 12$ its recall at $s_{\text{loc}} < 1\%$ was lower at every ef we tried. Running ACORN at one million vectors required fixing two heap overflows in its reference code: an upper-layer neighbor list was written one entry past its slot, and a per-level array was one element too short. The patch is part of our release and has been submitted upstream; AddressSanitizer reports no further errors.

Because the plans adapt their budgets, a plan's latency does not depend on the optimizer's estimate; the optimizer only chooses which plan runs. This makes the oracle well defined. A first generation of graph plans used in an earlier version of this work restarted a faiss search with a doubled budget and used faiss's bitmap-filtered HNSW as the fused plan. Table 15 compares the two generations; all other results use the second.

4.4 Measured Cost Model

Intuitively, a cost model answers "if the local selectivity were s, how long would each plan take?" A common shortcut is to count operations, such as distance computations. We instead fit each plan's cost to latencies measured on the target machine, because the same number of distance computations costs very different amounts in a sequential scan and in a random walk over a graph (Section 5.8 shows that counting makes routing 1.6–6.3× slower).

Staircase plans. A graph plan whose rounds have budgets $e_i = e_0 2^i$ and which stops once the budget reaches a need ν runs $j(\nu) = \lceil \log_2(\nu/e_0)\rceil^+$ rounds, capped at the budget limit. If a round of budget e costs $b_0 + b_1 e^\gamma$, the plan costs

$$\mathrm{St}(\nu; b_0, b_1, \gamma) = \sum_{i=0}^{j(\nu)} \big(b_0 + b_1 (e_0 2^i)^\gamma\big). \tag{6}$$

Plan costs. With $C_{\text{rel}} = c_0 + c_1 N|\mathcal{P}| + c_2 N\,\mathbb{I}[\text{join}]$ for columnar predicate evaluation,

$$C(\Pi_{\text{pre}}) = C_{\text{rel}} + a_0 + c_{\text{scan}}\, s_g N, \tag{7}$$

$$C(\Pi_{\text{post}}) = \mathrm{St}\big(k/s_{\text{loc}};\ \theta_{\text{post}}\big), \tag{8}$$

$$C(\Pi_{\text{fused}}) = C_{\text{rel}} + \mathrm{St}^{+1}\big(c_f (k/s_{\text{loc}})^{\beta_f};\ \theta_{\text{fused}}\big), \tag{9}$$

where St^{+1} adds the confirmation round. The need of Π_{post} is exactly $h_k = k/s_{\text{loc}}$ (Definition 3.1). Filtered traversal needs a budget that grows faster than h_k, which we capture with β_f.

Calibration. The constants are fitted on the training queries using their *true* selectivities and measured latencies: (c_0, c_1, c_2) and (a_0, c_{scan}) by non-negative least squares on relative error, and θ by nonlinear least squares in log space ($\gamma \in [0.3, 2.5]$, with c_f and β_f chosen on a grid). On SYN-128 the fit gives $c_1 = 0.53$ ns per row and predicate term and $c_{\text{scan}} = 16.0$ ns per vector for the NEON kernel. It also gives $\gamma_{\text{post}} = 1.25$, $\gamma_{\text{fused}} = 0.65$, and $\beta_f = 2.00$. Exponents above one are common: a graph search's priority queues make large budgets disproportionately expensive. With the first-generation plans, a single faiss round at $e = 65{,}536$ took about 2 s on SYN-128. A model restricted to $\gamma \le 1$ underpredicted those costs by up to 10× and misrouted tail queries even when given the true selectivities.

Counted-cost ablation. To show that costs must be measured rather than counted, we also fit the same staircases to *distance-evaluation counts* instead of time, so that a sequential and a random distance computation cost the same, as in cost models that count distance evaluations.

Example. At the true selectivities, the fitted model prices Query A at 3.54 ms for pre-filtering, 8.67 ms for fused traversal and 16.55 ms

for post-filtering (measured: 4.03, 1.43 and 17.25 ms), and Query B at 4.40, 4.05 and 0.16 ms (measured: 5.23, 2.58 and 0.11 ms). At these selectivities the model picks the oracle's plan for both queries. It overestimates ACORN on Query A, where ACORN is in fact fast but returns only 0.8 of the true answers; ACORN's cost is the hardest to model (Section 5.3).

4.5 Choosing a Plan Robustly

Intuitively, the estimate can be wrong by a factor of two or three, and a wrong estimate is costly in one direction only. Pre-filtering's cost does not depend on s_{loc} at all, while a graph plan's cost explodes when s_{loc} is smaller than believed. HVO therefore asks, for each plan, "how bad could this plan be if the truth were anywhere in my uncertainty interval?" and picks the plan whose worst case is best.

Point routing picks $\arg\min_{\Pi} C(\Pi; \hat{s}_g, \hat{s}_{\text{loc}})$. *Minimax routing* uses an interval $I = [\hat{s}_{\text{loc}}/\eta,\ \hat{s}_{\text{loc}}\eta]$ and picks

$$\begin{aligned}\hat{\Pi} &= \arg\min_{\Pi}\ \max_{s\in I} C(\Pi; \hat{s}_g, s) \\ &= \arg\min_{\Pi}\ \max\left(C(\Pi; \hat{s}_g, \hat{s}_{\text{loc}}/\eta),\ C(\Pi; \hat{s}_g, \hat{s}_{\text{loc}}\eta)\right),\end{aligned} \tag{10}$$

since every plan cost is monotone in s_{loc}. We set η to the 0.9-quantile of the estimator's Q-error on the training split, fixed before looking at test data ($\eta = 1.87$ and 2.10 for HPBS). Because $C(\Pi_{\text{pre}})$ does not depend on s_{loc}, minimax routing prefers the plan whose cost is certain whenever the estimate is uncertain.

Example. With the interval width fitted on the training queries ($\eta = 2.52$), Query B's interval is $\hat{s}_{\text{loc}} \in [4.93\%,\ 31.2\%]$. Even at its pessimistic end the post-filter is predicted to take 0.63 ms, against 4.43 ms for pre-filtering, so HVO post-filters. Query A's interval is $[0.023\%,\ 0.145\%]$. At its pessimistic end the post-filter could take 58.55 ms and fused traversal 8.67 ms, while pre-filtering costs 3.54 ms whatever the truth, so HVO pre-filters. Both are the oracle's choices.

Recall-aware routing. Graph plans are approximate. For each graph plan we fit a logistic model of $\Pr[\text{Recall@}k \geq 0.9]$ on the training split, using features available at planning time ($\ln \hat{s}_{\text{loc}}$, $\ln \hat{s}_g$, the number of terms, a join indicator). The recall-aware router admits a graph plan only if this probability is at least 0.9; otherwise it falls back to the exact pre-filter plan. This trades latency for recall, and we report both variants.

Native planner. The whole decision (Algorithm 2) runs in one C call, `hvo_plan`, shared by the benchmark and the PostgreSQL extension: sketch estimate, calibrator forward pass, recall check and minimax costing. The call takes the encoded predicate and the query vector and returns the plan.

Runtime guard. We also evaluated an early-exit rule that aborts Π_{post} after its first round if the observed match rate is below half the estimate and the cost model predicts Π_{pre} to be cheaper than the remaining rounds. It is replayed exactly from the recorded per-round timings.

4.6 Guarantees

The example shows minimax routing doing the right thing on two queries. The results below say when it is guaranteed to, and what an estimation error of a given size can cost. *Intuitively*, three facts drive them. A graph plan doubles its budget in rounds, so a small estimation error changes its cost by at most a few rounds

Table 5: Algorithm 2: HVO planning for one query (the native `hvo_plan` call).

Input: $\mathbf{q}$, k, $\mathcal{P}$; sketch; calibrator f_Θ; cost model $C(\cdot)$ with plans $\{\Pi_{\text{pre}}, \Pi_{\text{fused}}, \Pi_{\text{post}}\}$; interval width η; recall model $\hat{p}_\Pi$ and threshold τ (recall-aware variant only).
1: $(\hat{s}_g, \hat{s}^{\text{raw}}_{\text{loc}}, \mathbf{z}) \leftarrow$ median over the L partitions of Algorithm 1; $\mathbf{z}$ holds the 12 calibrator features.
2: $\hat{s}_{\text{loc}} \leftarrow \text{clip}(\hat{s}^{\text{raw}}_{\text{loc}}\, e^{f_\Theta(\mathbf{z})},\ k/N,\ 1)$.
3: **for** each plan Π: $\bar{C}(\Pi) \leftarrow \max\left(C(\Pi; \hat{s}_g, \hat{s}_{\text{loc}}/\eta),\ C(\Pi; \hat{s}_g, \hat{s}_{\text{loc}}\eta)\right)$ ▷ Eq. (10)
4: **if** recall-aware: **for** each graph plan Π with $\hat{p}_\Pi(\hat{s}_g, \hat{s}_{\text{loc}}, \mathcal{P}) < \tau$: $\bar{C}(\Pi) \leftarrow \infty$.
5: **return** $\hat{\Pi} \leftarrow \arg\min_\Pi \bar{C}(\Pi)$ ▷ point routing: $\eta = 1$

(Lemma 4.2). Pre-filtering's cost does not depend on s_{loc}, so a minimax router never gives up a query whose best plan is the exact scan (Theorem 4.4). And a router that sees only s_g cannot distinguish the two panels of Figure 2, so it can be made arbitrarily slow (Proposition 4.5).

Regret is $\mathcal{R} = C(\hat{\Pi}; s)/C(\Pi^*; s)$, where Π^* minimizes cost at the true selectivities $s = (s_g, s_{\text{loc}})$. The results below concern the cost model of Section 4.4 and the routers of Section 4.5. Measured regret differs from model regret by the model's calibration error, which we report in Section 5. All statements hold per query, given that query's estimation error.

Lemma 4.2 (Sensitivity of staircase plans). *Let a plan's cost be $A + \text{St}(c\,(k/s)^\beta; b_0, b_1, \gamma)$ with $A, b_0, b_1 \geq 0$ and $\gamma > 0$. If $\hat{s}/s \in [1/\eta, \eta]$, then its cost at $\hat{s}$ is within a factor $\Lambda_{\beta,\gamma}(\eta)$ of its cost at s, where*

$$\Lambda_{\beta,\gamma}(\eta) = 1 + \delta\, 2^{\gamma\delta}, \qquad \delta = \lceil \beta \log_2 \eta \rceil.$$

Proof. The needs differ by a factor of at most η^β, so the round counts differ by at most δ, because $\lceil x + y \rceil \leq \lceil x \rceil + \lceil y \rceil$. Suppose the larger cost has $j + \delta$ rounds and the smaller j (fewer additional rounds only shrink the ratio). The constant parts have ratio $(j + \delta + 1)/(j + 1) \leq 1 + \delta$. With $r = 2^\gamma$ the geometric parts have ratio

$$\frac{\sum_{i\leq j+\delta} r^i}{\sum_{i\leq j} r^i} = 1 + \frac{r^{j+1}}{\sum_{i\leq j} r^i} \sum_{i<\delta} r^i \leq 1 + r \sum_{i<\delta} r^i \leq 1 + \delta r^\delta,$$

using $\sum_{i\leq j} r^i \geq r^j$ and $r \geq 1$. A ratio of sums is at most the largest ratio of its parts, and adding $A \geq 0$ to both costs only decreases the ratio. Capping the round count at the budget limit also only decreases it. □

The factor is 1 when $\eta < 2^{1/\beta}$ *and* no doubling boundary lies between the two needs. In the worst case, though, an arbitrarily small error can cross a boundary, which costs one extra round and gives the factor $1 + 2^\gamma$. This discreteness is a real property of adaptive plans, not an artifact of the proof.

Let $\Lambda = \max(\eta_g, \Lambda_{1,\gamma_{\text{post}}}(\eta), \Lambda_{\beta_f,\gamma_{\text{fused}}}(\eta))$, where η_g bounds the multiplicative error of $\hat{s}_g$, which enters $C(\Pi_{\text{pre}})$ affinely.

Theorem 4.3 (Point routing). *If $\hat{s}_g/s_g \in [1/\eta_g, \eta_g]$ and also $\hat{s}_{loc}/s_{loc} \in [1/\eta, \eta]$, then point routing has $\mathcal{R} \leq \Lambda^2$. If, in addition, $\hat{s}_g = s_g$ and either $\hat{\Pi}$ or Π^* is Π_{pre}, then $\mathcal{R} \leq \Lambda$.*

Proof. By Lemma 4.2, $C(\Pi; \hat{s}) \in [C(\Pi; s)/\Lambda,\ \Lambda C(\Pi; s)]$ for every plan. Hence $C(\hat{\Pi}; s) \leq \Lambda C(\hat{\Pi}; \hat{s}) \leq \Lambda C(\Pi^*; \hat{s}) \leq \Lambda^2 C(\Pi^*; s)$. If $\hat{\Pi} = \Pi_{\text{pre}}$ and its cost is exact, the first factor is 1. If $\Pi^* = \Pi_{\text{pre}}$, the last factor is 1. □

Theorem 4.4 (Minimax routing). *Suppose $\hat{s}_g = s_g$ and $s_{loc} \in I = [\hat{s}_{loc}/\eta, \hat{s}_{loc}\eta]$. Then minimax routing (Eq. (10)) satisfies: (i) if $\Pi^* = \Pi_{pre}$, then $\mathcal{R} = 1$; (ii) in all cases $\mathcal{R} \le \Lambda(\eta^2)$.*

Proof. Let $\bar{C}(\Pi) = \max_{s' \in I} C(\Pi; s')$; since $s_{\text{loc}} \in I$, $C(\Pi; s) \le \bar{C}(\Pi)$. (i) $C(\Pi_{\text{pre}})$ does not depend on s_{loc}, so $\bar{C}(\Pi_{\text{pre}}) = C(\Pi_{\text{pre}}; s)$, and $C(\hat{\Pi}; s) \le \bar{C}(\hat{\Pi}) \le \bar{C}(\Pi_{\text{pre}}) = C(\Pi^*; s)$. (ii) $C(\hat{\Pi}; s) \le \bar{C}(\hat{\Pi}) \le \bar{C}(\Pi^*) \le \Lambda(\eta^2) C(\Pi^*; s)$, because any $s' \in I$ is within a factor η^2 of s_{loc}. □

Theorem 4.4 is the formal reason for minimax routing. Misrouting a query whose best plan is the exact pre-filter scan to a graph plan is the failure that produces the worst tail latencies: the graph plan can be orders of magnitude slower. Minimax routing provably avoids it whenever the interval covers the truth. The price is conservatism: when the interval is wide, the router may scan where a graph plan would have been faster.

Proposition 4.5 (Global-selectivity routing). *Let a router choose its plan as any function of s_g alone. Let graph-plan costs satisfy $C_G(1) \le C_1$ and $C_G(k/N) \ge bN^\gamma$ for constants $C_1, b, \gamma > 0$, and let $C(\Pi_{pre}) = a + c' s_g N$, which holds when predicates are evaluated through an index at a cost proportional to their output. Then for N large enough that $k \le \sqrt{bN^\gamma C_1/c'}$, its worst-case regret over instances with that N is at least $\sqrt{bN^\gamma/(c'C_1)} / (1 + o(1))$, that is, $\Omega(N^{\gamma/2})$.*

Proof. Fix $s^* = \sqrt{bN^\gamma C_1/c'}/N \ge k/N$. Matching tuples can be placed around $\mathbf{q}$ ($s_{\text{loc}} = 1$) or beyond all non-matching tuples ($s_{\text{loc}} = k/N$) while keeping $s_g = s^*$, and the router cannot tell the two instances apart. If it picks Π_{pre}, the first instance gives $\mathcal{R} \ge c' s^* N / C_1$. If it picks a graph plan, the second gives $\mathcal{R} \ge bN^\gamma/(a + c' s^* N)$. Both equal $\sqrt{bN^\gamma/(c'C_1)}$ up to the lower-order term a. □

With columnar predicate evaluation, $C(\Pi_{\text{pre}}) \ge c_1 N$ for every s_g. The same argument then gives regret $\Omega(\min(N, N^{\gamma-1}))$. This is a constant set by the ratio of random-access to per-row predicate cost when $\gamma \le 1$, and it still grows with N for the superlinear γ we measure. HVO's bounds, by contrast, depend only on the estimation error. This contrast is why we measure the estimator's tail and not only its median.

4.7 Integration into PostgreSQL

We implemented HVO as a PostgreSQL 16 extension (`hvo_pg`) for `pgvector` 0.8.7, using `planner_hook`. For a query of the form of Eq. (1) with `ORDER BY embedding <#> q LIMIT k`, the hook does the following:

- It reads $\mathbf{q}$ and k from the parse tree, together with range, equality and `IN` predicates on the fact table and the semi-join `fk IN (SELECT id FROM dims WHERE . . . )`. Casts and constant expressions are folded.
- It encodes the predicate against the sketch and evaluates the dimension predicate on an in-memory copy of the dimension columns.
- It calls `hvo_plan` and steers the standard planner with plan-type settings for the duration of planning, as `pg_hint_plan` does: turning `enable_indexscan` off for the pre-filter plan, and turning `enable_seqscan` and `enable_bitmapscan` off for `pgvector`'s iterative HNSW scan.

`pgvector` has no in-traversal filtering, so routing is two-way. The model file holds the sketch, the calibrator, the interval width and a cost and recall model. Both models are fitted to PostgreSQL's own execution times and recall on the training queries, using forced pre-filter and forced iterative-scan runs. The hook is about 660 lines of C plus the shared planner code.

Example. Query B reaches PostgreSQL as

```
SELECT id FROM items WHERE published IN (1)
  AND archive IN (4,6,9,14,21,22,27,28,30)
  ORDER BY embedding <#> '[...]' LIMIT 10
```

where the literal holds the 384 coordinates of the query paper's embedding. The hook reads the vector, the two predicates and the limit, calls the planner, and receives Π_{post}; it then lets PostgreSQL plan with sequential and bitmap scans disabled, so `pgvector`'s iterative HNSW scan is chosen. For Query A it receives Π_{pre} and disables index scans, so PostgreSQL filters first and sorts the matches by distance. Queries the hook does not recognize are planned normally.

4.8 Assumptions and Implementation

The method rests on the following assumptions, each of which the evaluation tests or the discussion revisits.

- Vectors are unit-norm and similarity is the inner product (equivalently cosine), as in all four datasets.
- Within a sketch cell, predicate terms are independent of one another and of a tuple's position in the cell (Section 4.2). Correlation *across* cells is captured; the within-cell residual is the error term β of Proposition 4.1.
- Every plan adapts its budget until it has found k matches, so a plan's latency does not depend on the estimate. This makes the oracle well defined.
- Plan costs are monotone in s_{loc}, which Eq. (10) and Theorems 4.3 and 4.4 require.
- Cost constants, the calibrator, η, the recall model and the thresholds of the global routers are fitted on the training split only, separately for each machine, scan kernel and plan set.

Implementation. The scan kernels, the resumable HNSW scan and the planner (HPBS estimation, calibrator, recall check and minimax costing) are written in C and called from Python through `ctypes`. ACORN is linked through a small C wrapper, and the PostgreSQL extension shares the planner source. The experiment drivers, estimator baselines and analysis are Python. Algorithms 1 and 2 and the parameter values in Table 6 are sufficient to reproduce the method; the released code reproduces every number in Section 5.

5 Evaluation

We ask four questions. How accurate and how fast is HPBS, compared with histogram, exact-count, sampling and index-probing estimators (Section 5.2)? Does better estimation translate into lower end-to-end latency (Section 5.3)? How do the scan kernel and each component matter (Section 5.8)? And how does measured regret compare with the bounds (Section 5.9)? Every plan is executed for every test query on real hardware, so the best plan is measured, not modeled. Section 6 discusses limitations.

5.1 Evaluation Settings

5.1.1 Hardware and Software. Apple M4 (4 performance and 6 efficiency cores, 16 GB unified memory), macOS 26.3; Python 3.14.3 with NumPy 2.4.3 (Accelerate BLAS), SciPy 1.17.1 and scikit-learn 1.8.0 (calibrator and recall model); faiss-cpu 1.13.2; sentence-transformers 5.5.1 on PyTorch 2.12.0 for the embeddings; Apple clang 17.0.0 with `-O3 -mcpu=native` for the C code; PostgreSQL 16.14 with `pgvector` 0.8.7. All latencies are measured on one thread (OpenMP and Accelerate both limited to one thread). Each plan is run once as a warm-up and then twice; we report the slower of the two timed runs. We discarded two runs. A first run of the 768-d benchmark swapped, because two copies of the vectors (table and index) exceeded memory. Another run overlapped an unrelated job on the machine. In the reported runs the table aliases the index's vector storage, no other heavy job ran, and we logged swap activity every minute. The remaining swap-ins came from other processes paging in, and the timed process stayed CPU-bound. PostgreSQL runs on the same machine with `shared_buffers`=4 GB, parallel query and JIT disabled, and `pgvector`'s default HNSW parameters ($m = 16$, $ef_{\text{construction}} = 64$). PostgreSQL latencies are client wall-clock times and therefore include planning.

5.1.2 Datasets. Synthetic data. Each synthetic benchmark has $N = 10^6$ unit-norm vectors from a two-level mixture (64 topics × 16 subtopics with Zipfian popularity) in $d = 128$ (SYN-128) or $d = 768$ (SYN-768). Attributes are generated from the latent topic with graded correlation. `category` (32 values) equals the topic's value with probability 0.7; `lang` (12 values) follows the subtopic with probability 0.5; `year` has a topic-specific mean plus Gaussian noise; `rating` has a weak subtopic effect; `price` (log-normal) and `views` (Pareto) are independent of the vectors. A foreign key references a 20,000-row seller table with `region` (10 values) and `tier` (4 values). For 60% of rows the seller is drawn, with Zipfian popularity, from the topic's preferred region.

Real data (ARXIV-384). We sample 10^6 papers, plus a disjoint pool of 3000 query papers, from the CC0-licensed arXiv-abstracts-2021 dump (about 2×10^6 papers) [30]. Each paper is embedded with the sentence encoder all-MiniLM-L6-v2 [33] ($d = 384$, title and abstract truncated to the model's 128-token training length, unit norm). All attributes are real metadata:

- `archive`: the top-level archive of the primary category (31 archives plus "other");
- `published`: whether the paper has a journal reference;
- `year`: the submission year, taken from the identifier (1991–2021);
- `n_cats`: the number of listed categories;
- `n_authors` and `abs_words`: the author count and abstract length.

A foreign key references a dimension table of the 172 primary categories, with their subject `group` (8 values: physics, mathematics, computer science, …) and a popularity quartile. Spatial-relational correlation is not injected here: it arises because a paper's text determines both its embedding and its field.

Real data (AMAZON-384). From the Amazon Reviews 2023 item metadata [31] for four departments (video games, musical instruments, baby products, industrial and scientific; about 10^6 products), we keep 990,000 products and a disjoint pool of 3000 query products. Each product is embedded the same way, from its title, features and description. Its real attributes are the department, the main and second-level category, the price (0 when no price is listed: a point mass covering 57% of the products), the average rating, the number of ratings, the number of listed features and the description length. A foreign key references a table of about 144,000 stores, with their size and quality quartiles. Here predicates are *anchored on the query product*, as a shopper would filter: the same department (80%) or another one (20%, a cross-category recommendation, which gives anti-correlated queries), a price band around the product's price, a minimum rating, a popularity threshold, the same main category, or a seller tier. No public query log exists for filtered vector search, so this workload is derived rather than logged. We release both benchmarks, with their predicates, k-th-match selectivities and exact ground truth at $k \in \{1, 10, 100\}$ [32].

5.1.3 Query Workloads. On synthetic data a query vector is a data vector plus Gaussian noise of norm about 0.35, renormalized; on the real datasets it is the embedding of a held-out item. On SYN-128, SYN-768 and ARXIV-384 the predicate is a conjunction of 1–3 terms drawn independently of the vector: numeric ranges of random width 2–90% of the quantile range, `IN` lists, and semi-joins on the two dimension attributes. Because predicates are drawn independently of the query, correlation arises naturally: a `category` or `archive` filter either includes or excludes the query's topic or field. We generate 1000 training and 1500 test queries per benchmark from disjoint seeds and drop queries with no match. The test splits contain 630/427/413 (ARXIV-384), 258/329/868 (AMAZON-384), 640/605/255 (SYN-128) and 647/605/248 (SYN-768) anti-correlated/independent/positively correlated queries. They include 164, 57, 56 and 60 queries with $s_{\text{loc}} < 10^{-3}$. The anchored AMAZON-384 predicates are mostly positively correlated. We evaluate $k \in \{1, 10, 100\}$, with $k = 10$ unless stated. Ground truth (exact s_g, h_k, and filtered top-10) comes from brute force. Everything fitted—the calibrator, the cost model, the thresholds of the global routers, and the interval width η—uses only the training split.

5.1.4 Baselines. The estimators are:

- *1-D histograms + AVI* (PostgreSQL-style): 100 equi-depth bins per continuous column, a most-common-value list for continuous columns with point masses, and exact value frequencies for categorical and integer-valued columns, independence across terms, a uniform foreign-key assumption, and $\hat{s}_{\text{loc}} = \hat{s}_g$.
- *Exact s_g + AVI* (Milvus-style): the exact predicate count, with $\hat{s}_{\text{loc}} = s_g$.
- *1% uniform sample*: the rank of the first sampled match among sampled tuples.
- *HNSW probe* (Exqutor-style): the match rate among the first 256 unfiltered neighbors returned by the index ($ef = 256$).

Each is fed to the same cost model, with point and with minimax routing. The routers additionally include the three static plans and

global-selectivity routers with two-way ($\Pi_{\text{pre}}/\Pi_{\text{post}}$) and three-way thresholds on the exact s_g, tuned on the training split to minimize mean latency (which is generous to them). Inside PostgreSQL we compare the following configurations on the same SQL queries:

- `pgvector` with its default settings;
- `pgvector` with its iterative scan (`strict_order`), its recommendation for filtered queries;
- **Exqutor** [18], built from its released code (`pgvector` 0.7.1 and PostgreSQL 16.9 with its patches; a missing prototype had to be added to compile with clang);
- forced pre-filter and forced iterative post-filter plans;
- the HVO hook, with and without the recall constraint.

Exqutor's estimator only activates for vector *range* predicates in queries over more than two tables, so on these top-k queries it plans exactly like `pgvector` 0.7.1. We include it because the comparison was requested, not because it targets this workload. We do not compare against Milvus or DuckDB as systems.

5.1.5 Metrics. The oracle runs all three plans and picks the fastest with Recall@$k \geq 0.9$; in PostgreSQL it takes the faster of the two forced plans that meets the target; the other routers are charged their plan's latency plus their planning time. Measured regret is the ratio of a router's latency to the oracle's. Plan accuracy is agreement with the oracle's choice, which penalizes near-ties, so we emphasize latency and regret.

5.1.6 Parameters, Seeds and Runs. Table 6 lists every parameter. The four benchmarks use seeds 11, 22, 33 and 44 (SYN-128, SYN-768, ARXIV-384, AMAZON-384); training and test predicates are drawn with that seed plus 1000 and plus 2000 (AMAZON-384's anchored predicates are drawn during data preparation with its seed), partition ℓ of the sketch uses seed $100 \cdot \text{seed} + \ell$, and the calibrator uses `random_state=0`. The whole pipeline (data, sketches, ground truth, plan runs, analysis) was run once per benchmark and per k. The experimental unit is the query: every test query is executed with every plan, giving 1,455–1,500 paired observations per benchmark.

5.1.7 Statistical Analysis. We report 95% percentile-bootstrap confidence intervals with 2000 resamples of test queries (fixed seed). For a difference between two routers we resample queries *paired*, so both routers are evaluated on the same resampled queries, and report the interval of the relative gain $1 - \overline{\text{lat}}_{\text{HVO}}/\overline{\text{lat}}_{\text{global}}$. The intervals capture query-sampling variability. They do not capture run-to-run timing noise, which we did not replicate. Re-running the analysis, which re-measures planning time, moved mean latencies by at most 0.003 ms. We make no claim of significance beyond these intervals and perform no multiple-comparison correction.

Table 6: Parameters used in all experiments.

Component	Setting
HPBS	$L = 4$ partitions, $m = 1024$ k-means cells (12 iterations on 2×10^5 vectors), 16 radial quantiles, $B = 32$ bins per numeric attribute, $k_{\text{kmv}} = 64$, 26 bisection steps
Calibrator	MLP $12 \to 16 \to 16 \to 1$ (497 parameters), ReLU, L_2 10^{-3}, early stopping on 15% of training queries
HNSW (faiss)	$M = 32$, $ef_{\text{construction}} = 200$, inner product
Π_{post}	resumable scan, window $ef = \max(64, k)$, batches $k, k, 2k, 4k, \ldots$, budget 2^{17}
Π_{fused} (ACORN-1)	$M = 32$, $M_\beta = 64$, $\gamma = 1$, ef doubled from 64 until stable
Π_{pre}	scalar FP32, NEON FP32, NEON INT8 (top $4k$ reranked in FP32)
Baselines	100 equi-depth bins + MCV list; 1% uniform sample; HNSW probe of 256 neighbors ($ef = 256$)
Routing	η = 0.9-quantile of training Q-error; recall threshold $\tau = 0.9$ on Pr[Recall@$k \geq 0.9$]
Workload	1000 training and 1500 test queries, 1–3 predicate terms, $k \in \{1, 10, 100\}$
Oracle	fastest plan with Recall@$k \geq 0.9$
PostgreSQL	`shared_buffers`=4 GB; `pgvector` $m = 16$, $ef_{\text{construction}} = 64$, iterative scan `strict_order`
Statistics	paired percentile bootstrap, 2000 resamples, 95%

5.2 Estimation Accuracy

Table 7 and Figure 5 report Q-error on the test split.

Independence-based estimators fail under correlation, worst on real data. Histogram-based and exact-count AVI reach P99 Q-errors of 765.57 and 514.26 on ARXIV-384 and 256.94 and 184.27 on AMAZON-384, against 25.96 and 26.21 on SYN-128. Their errors concentrate in the anti-correlated regime and on rare local matches. Knowing s_g exactly does not help, because the error is in assuming $s_{\text{loc}} = s_g$. The exact count also costs 1496 μs of planning time.

Sampling starves. The 1% sample (P99 18.41 and 21.83 on the real data) bases its estimate on very few sampled matches near the query.

Index probing is accurate where matches are dense and fails where they are rare. The HNSW probe is the strongest baseline. Its median Q-error is the lowest of all estimators on every dataset (e.g. 1.21 on ARXIV-384), because it observes the query's neighborhood directly. When no match appears among the first 256 neighbors it can only report an upper bound, so its P99 is 39.23 on ARXIV-384 and its rare-match P99 is 134.39 there. It also costs 221 μs of random index accesses during planning.

HPBS. The calibrated sketch has the lowest P99 Q-error on ARXIV-384 (6.17), SYN-128 (3.08) and SYN-768 (4.31), and an HPBS variant has the lowest rare-match P99 on all three (15.59 against 134.39 for the probe on ARXIV-384). AMAZON-384 is the exception: the probe is slightly more accurate there (P99 14.86 against 16.31). Its anchored predicates make matches crowd right around the query product. That is within-cell spatial-relational correlation, the error term β of Proposition 4.1 that the sketch averages away and the probe observes. The calibrator mainly removes the sketch's tail: the P99 falls from 9.95 to 6.17 on ARXIV-384 and from 29.86 to 16.31 on AMAZON-384. On the real data one partition ($L = 1$) is about as accurate after calibration (6.19 and 16.83) at less than half the planning time; on synthetic data it is less accurate. Cross-polytope cells, our original design, are the least accurate variant everywhere.

Point masses. AMAZON-384 also exposed a histogram failure. 57% of products have price 0, and equi-depth bins collapse onto that value. Before we gave point masses their own bins (Section 4.2), the calibrated sketch's P99 Q-error on AMAZON-384 was 82. The same fix, as a most-common-value list, is applied to the histogram baseline.

Table 7: Local-selectivity Q-error on the test split, by correlation regime and for rare local matches ($s_{\text{loc}} < 10^{-3}$), and median planning-time cost of the estimator. Lower is better.

	Anti-correlated			Independent			Positively correlated			Rare	Time
Estimator	P50	P95	P99	P50	P95	P99	P50	P95	P99	P99	(μs)
ARXIV-384 (real)											
1-D hist. + AVI (PG-style)	7.49	459.83	1265.85	1.23	3.42	17.65	1.67	7.31	24.04	2002.20	3
Exact s_g, s_{loc}=s_g (Milvus-style)	6.23	238.56	1133.30	1.21	1.63	1.76	1.71	5.43	18.67	1312.87	1496
Uniform sample 1%	2.80	10.32	30.67	1.80	6.27	11.58	1.30	3.82	7.97	53.68	255
HNSW probe, P=256 (Exqutor-style)	1.46	17.95	45.71	1.16	1.89	18.39	1.12	1.75	2.51	134.39	221
HPBS, cross-polytope cells	5.26	55.64	151.79	6.77	21.34	26.40	21.23	31.46	36.74	287.10	135
HPBS raw (L=4, m=1024)	1.44	4.63	8.07	1.97	5.23	6.41	4.67	8.76	11.02	23.29	134
HPBS, L=1 + calibrator	1.54	4.02	7.80	1.24	2.16	2.78	1.51	3.42	5.36	13.87	57
HPBS + calibrator	1.52	4.02	7.03	1.28	2.23	3.39	1.53	2.95	6.19	15.59	138
AMAZON-384 (real)											
1-D hist. + AVI (PG-style)	3.56	120.43	418.10	1.30	3.60	223.54	3.91	37.19	160.15	678.47	9
Exact s_g, s_{loc}=s_g (Milvus-style)	3.18	146.50	348.08	1.18	1.65	1.76	3.58	25.57	75.94	504.03	1462
Uniform sample 1%	2.62	10.30	29.04	1.64	8.31	18.49	1.41	7.40	20.88	27.81	243
HNSW probe, P=256 (Exqutor-style)	1.43	8.82	39.38	1.17	3.58	20.96	1.19	2.39	5.32	142.99	171
HPBS, cross-polytope cells	3.69	36.79	71.00	8.03	36.49	151.81	21.75	42.86	227.74	904.05	135
HPBS raw (L=4, m=1024)	1.54	6.28	10.52	2.38	9.32	38.81	4.53	11.88	45.50	54.24	128
HPBS, L=1 + calibrator	1.85	9.46	17.20	1.40	5.78	22.71	1.49	5.36	9.70	30.76	55
HPBS + calibrator	1.80	7.17	20.98	1.49	6.05	25.44	1.52	4.48	10.65	36.24	132
SYN-128											
1-D hist. + AVI (PG-style)	3.01	12.16	73.43	1.24	1.74	1.89	2.19	8.15	13.46	169.62	11
Exact s_g, s_{loc}=s_g (Milvus-style)	2.97	14.02	73.38	1.23	1.68	1.78	2.20	8.19	11.13	169.67	1517
Uniform sample 1%	2.22	11.20	28.69	1.90	9.10	18.98	1.59	6.40	14.14	133.81	80
HNSW probe, P=256 (Exqutor-style)	1.27	3.69	6.73	1.16	2.17	3.24	1.16	1.71	2.56	25.53	113
HPBS, cross-polytope cells	1.79	10.96	27.67	3.37	18.81	23.98	14.31	30.87	36.26	59.82	124
HPBS raw (L=4, m=1024)	1.24	2.33	4.89	1.68	5.70	8.59	3.35	9.75	13.54	1.76	82
HPBS, L=1 + calibrator	1.32	2.44	3.72	1.25	2.09	3.20	1.53	3.95	7.20	2.38	43
HPBS + calibrator	1.29	2.13	2.85	1.26	1.99	2.77	1.41	2.65	3.96	2.30	86
SYN-768											
1-D hist. + AVI (PG-style)	2.98	14.77	58.43	1.24	1.76	1.93	2.28	7.20	18.72	136.61	11
Exact s_g, s_{loc}=s_g (Milvus-style)	2.96	16.38	58.94	1.23	1.72	1.80	2.30	6.83	19.01	139.95	1520
Uniform sample 1%	2.22	11.33	32.87	1.82	9.34	20.42	1.60	6.55	15.31	10.88	529
HNSW probe, P=256 (Exqutor-style)	1.27	4.67	7.71	1.16	1.92	3.76	1.17	1.88	2.27	12.97	169
HPBS, cross-polytope cells	1.92	10.79	22.05	3.41	18.90	27.48	11.76	29.96	35.77	58.01	134
HPBS raw (L=4, m=1024)	1.25	2.78	6.90	1.81	8.39	13.51	3.66	13.76	18.60	1.86	136
HPBS, L=1 + calibrator	1.35	2.60	4.75	1.25	2.06	2.45	1.60	5.17	8.49	2.20	51
HPBS + calibrator	1.33	2.34	3.55	1.26	1.96	2.75	1.60	4.20	5.99	2.08	140

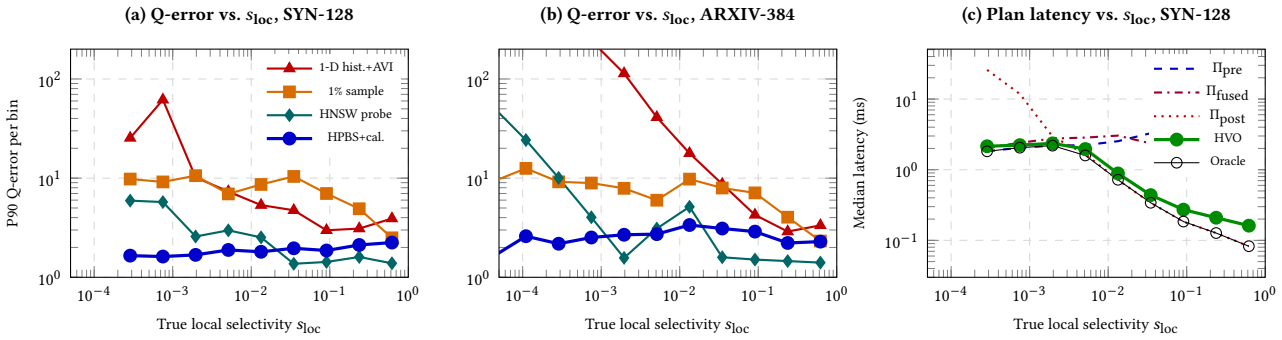


Figure 5: (a, b) P90 Q-error within bins of true local selectivity: the baselines degrade as matches become rare near the query, and HPBS does not. (c) Median latency of each plan by true local selectivity on SYN-128, with HVO (including planning time) and the oracle. Pre-filtering is flat in s_{loc}; the graph plans' cost explodes as s_{loc} falls.

5.3 End-to-End Latency

Tables 8 and 9 report latency, recall and regret for every router at $k = 10$. Figure 6 shows the full latency distributions on the real data, Figure 7 which plans each router chooses, and Figure 5(c) where the plans cross on SYN-128.

No static plan is acceptable. Pre-filtering always pays the full predicate scan (mean 14.94 ms on ARXIV-384). The resumable post-filter is the best plan for most queries, but rare local matches push its P99 to 110.80 ms. ACORN-1 is rarely fastest and loses recall at low local selectivity (mean recall 0.619 when used for every SYN-128 query).

Table 8: End-to-end routing on the real datasets ($k = 10$): latency (ms, including planning time), mean Recall@10, share of queries with Recall@10 < 0.9, plan accuracy against the oracle, measured regret, and median planning time (μs). CM: the calibrated cost model of Section 4.4; "robust": minimax routing with the interval width fitted on the training split.

Router	Mean	P50	P95	P99	Recall	Low (%)	Acc. (%)	Mean reg.	P99 reg.	Plan (μs)
ARXIV-384 (real)										
Always pre-filter	14.94	11.20	36.57	41.30	1.000	0.0	18.0	82.59	395.69	0
Always post-filter (resumable)	7.83	0.26	65.50	110.80	0.966	6.2	76.7	4.05	62.10	0
Always fused (ACORN-1)	5.40	2.54	16.13	42.59	0.930	14.5	5.2	15.64	91.12	0
Global s_g threshold	2.49	0.28	5.98	37.08	0.989	2.3	79.9	2.12	22.82	0
CM + 1-D histograms	4.78	0.27	25.11	97.70	0.978	4.7	81.6	2.26	30.80	3
CM + HNSW probe	1.55	0.56	5.89	9.83	0.984	3.1	89.4	1.98	4.28	221
CM + HNSW probe, robust	1.62	0.56	6.83	10.43	0.987	2.4	86.9	2.06	4.61	221
HVO, point routing	1.34	0.40	5.57	9.32	0.985	2.9	91.3	1.54	2.79	138
HVO (robust)	1.41	0.40	6.07	9.19	0.988	2.4	89.9	1.58	3.21	138
HVO, recall-aware	1.47	0.40	6.40	9.91	0.991	1.7	90.1	1.60	3.38	138
CM + true selectivities	1.14	0.26	5.10	8.10	0.987	2.5	92.7	1.01	2.01	0
Oracle	1.28	0.27	5.31	10.36	0.992	0.0	100.0	1.00	1.00	0
AMAZON-384 (real)										
Always pre-filter	12.63	8.15	38.37	42.79	0.999	0.0	10.8	89.34	459.48	0
Always post-filter (resumable)	2.69	0.19	9.45	74.40	0.977	4.3	87.5	1.99	29.75	0
Always fused (ACORN-1)	7.55	2.16	7.95	38.27	0.903	16.2	1.7	14.74	70.55	0
Global s_g threshold	1.11	0.20	2.82	18.65	0.983	3.5	87.6	1.47	13.71	0
CM + 1-D histograms	1.92	0.21	3.95	38.32	0.981	3.9	86.6	1.98	28.36	9
CM + HNSW probe	0.90	0.39	3.06	6.97	0.987	2.5	93.9	1.97	3.47	171
CM + HNSW probe, robust	0.93	0.39	3.24	7.40	0.989	2.3	93.0	2.00	3.63	171
HVO, point routing	0.97	0.35	3.24	7.16	0.985	3.0	93.3	1.92	3.38	132
HVO (robust)	0.96	0.36	3.30	7.00	0.988	2.5	91.1	2.00	6.54	132
HVO, recall-aware	1.01	0.36	3.62	7.92	0.989	2.2	90.7	2.05	6.85	132
CM + true selectivities	0.62	0.19	2.57	6.09	0.986	3.0	95.1	0.99	1.21	0
Oracle	0.86	0.20	3.08	10.16	0.994	0.0	100.0	1.00	1.00	0

Table 9: End-to-end routing on the synthetic datasets ($k = 10$); columns as in Table 8.

Router	Mean	P50	P95	P99	Recall	Low (%)	Acc. (%)	Mean reg.	P99 reg.	Plan (μs)
SYN-128										
Always pre-filter	4.53	3.66	10.31	10.82	1.000	0.0	12.4	34.05	202.46	0
Always post-filter (resumable)	1.28	0.23	5.24	13.41	0.976	3.9	85.9	1.36	9.62	0
Always fused (ACORN-1)	5.39	2.21	12.96	64.01	0.619	42.5	1.7	20.74	249.05	0
Global s_g threshold	0.72	0.23	2.40	5.69	0.978	3.3	91.2	1.09	4.85	0
CM + 1-D histograms	0.96	0.24	3.25	12.40	0.977	3.7	87.5	1.24	6.02	11
CM + HNSW probe	0.77	0.37	2.58	5.04	0.981	2.8	91.9	1.64	3.06	113
CM + HNSW probe, robust	0.76	0.37	2.59	3.39	0.982	2.7	90.4	1.67	3.37	113
HVO, point routing	0.73	0.42	2.59	3.41	0.980	2.8	94.1	1.59	3.22	86
HVO (robust)	0.73	0.42	2.52	3.21	0.981	2.7	93.2	1.59	3.27	86
HVO, recall-aware	0.73	0.42	2.52	3.24	0.981	2.7	93.1	1.60	3.27	86
CM + true selectivities	0.58	0.23	2.30	3.01	0.980	2.8	94.2	0.99	1.38	0
Oracle	0.67	0.24	2.34	4.19	0.992	0.0	100.0	1.00	1.00	0
SYN-768										
Always pre-filter	20.25	12.07	65.81	77.77	1.000	0.0	12.9	90.27	682.43	0
Always post-filter (resumable)	2.23	0.44	8.28	39.57	0.961	7.3	82.1	1.49	15.86	0
Always fused (ACORN-1)	18.67	2.49	23.85	76.82	0.591	45.9	5.1	15.92	247.70	0
Global s_g threshold	1.20	0.44	3.51	7.40	0.965	6.3	87.1	1.10	5.52	0
CM + 1-D histograms	1.40	0.44	4.69	17.88	0.963	6.7	86.3	1.13	4.60	11
CM + HNSW probe	1.37	0.70	4.00	6.47	0.966	6.1	89.4	1.55	2.87	169
CM + HNSW probe, robust	1.37	0.70	4.24	7.34	0.968	5.9	88.1	1.58	3.18	169
HVO, point routing	1.18	0.63	3.66	5.33	0.966	5.9	91.2	1.46	2.79	140
HVO (robust)	1.19	0.63	3.62	5.57	0.967	5.9	90.3	1.46	2.79	140
HVO, recall-aware	1.29	0.63	3.77	8.57	0.967	5.9	89.4	1.52	3.07	140
CM + true selectivities	0.98	0.44	3.28	5.06	0.967	5.9	91.1	0.96	1.35	0
Oracle	1.45	0.45	4.12	15.19	0.989	0.0	100.0	1.00	1.00	0

On real data HVO lowers tail latency, and on ARXIV-384 also mean latency. On ARXIV-384, HVO averages 1.41 ms, against 2.49 ms for the best tuned global-selectivity router (43% lower; paired 95% interval 34–51%), and its P99 latency is 9.19 ms against 37.08 ms. On AMAZON-384 the mean is 13% lower (0.96 vs. 1.11 ms), but the interval (−4 to 28%) includes zero, so this mean gain is not established at our sample size; the tail gain is clear (P99 7.00 vs. 18.65 ms, Figure 6b). Recall is the same as the global router's (0.988 vs. 0.989 on ARXIV-384), and HVO is close to the cost model fed true selectivities (1.14 and 0.62 ms). The oracle's mean (1.28 and 0.86 ms) is sometimes *above* HVO's, because it only accepts plans with Recall@10 ≥ 0.9.

On synthetic data HVO ties the global router on the mean (both intervals include zero, Figure 9) and wins on the tail. HVO averages 0.73 and 1.19 ms against 0.72 and 1.20 ms, with P99 3.21 vs. 5.69 ms and 5.57 vs. 7.40 ms. The resumable post-filter is strong across most

Table 10: Inside PostgreSQL 16 + `pgvector` on the real datasets ($k = 10$, test split): client latency in ms (including planning), mean Recall@10, and share of queries with Recall@10 < 0.9. The oracle takes the faster forced plan that meets Recall@10 ≥ 0.9 (pre-filtering if neither does). Exqutor was run on ARXIV-384 only; it needs its own patched PostgreSQL build and a separate copy of the data.

	ARXIV-384					AMAZON-384				
Configuration	Mean	P50	P99	Recall	Low (%)	Mean	P50	P99	Recall	Low (%)
pgvector 0.8.7, defaults	0.60	0.51	1.47	0.540	56.8	0.46	0.27	8.03	0.686	40.5
pgvector 0.8.7, iterative scan	5.09	0.60	27.21	0.846	28.1	2.20	0.35	26.24	0.904	16.8
Exqutor (pgvector 0.7.1 + patches)	0.30	0.28	0.49	0.538	56.1	–	–	–	–	–
Forced pre-filter	115.86	139.62	189.00	1.000	0.0	90.29	79.49	213.70	0.999	0.0
Forced iterative post-filter	18.04	0.67	214.62	0.907	20.7	5.69	0.41	153.83	0.931	14.1
HVO hook	16.10	0.80	172.03	0.953	10.5	4.45	0.64	51.04	0.953	9.4
HVO hook, recall-aware	50.00	1.21	184.58	0.984	3.5	43.64	4.97	229.54	0.988	2.2
Oracle	24.17	0.71	181.95	0.986	0.0	10.16	0.41	164.87	0.991	0.0

of the selectivity range, so a single threshold already routes most queries well. The mean gain then comes only from the minority of queries the threshold misroutes, and HVO's planning time offsets it. With one partition ($L = 1$, Table 12) HVO is faster than the global router on all four datasets, and all four paired intervals exclude zero (gains 48, 25, 10 and 11%; lower ends 39, 10, 2 and 4%).

Which queries are routed differently. Figure 7 shows that HVO and the global router send similar shares of queries to each plan. The difference lies in which queries go where, and the latency distributions (Figure 6) show where it pays: both routers match the fast post-filter on most queries, but HVO avoids the misrouted rare-match queries that make up the global router's tail.

Back to the running example. The pair of Figure 1 was selected to illustrate the failure mode, so it is fair to ask how often the situation arises. On the ARXIV-384 test queries, HVO and the best global router choose different plans for 15% of queries, and on those queries HVO's choice (including its planning time) is faster 89% of the time. The global router's plan differs from the oracle's on 20% of queries and HVO's on 10%. On AMAZON-384 the routers disagree less often (9%), HVO is faster on 70% of those queries, and the error rates are 12% and 9%, which is consistent with the smaller and less certain mean gain there.

Correlation-blind estimates poison a good cost model, and index probing is the closest competitor. Fed histogram estimates, the same cost model averages 4.78 ms on ARXIV-384. Fed the HNSW probe, it averages 1.55 and 0.90 ms (1.62 and 0.93 ms with minimax routing). That is behind HVO on ARXIV-384, where rare matches dominate the tail, and slightly *ahead* of HVO on AMAZON-384 (0.96 ms), where the probe estimates more accurately; HVO with $L = 1$ (0.83 ms) is ahead of both. HVO's structural advantages over probing are that it never touches the index during planning and that it remains accurate on rare matches.

Recall-aware routing. Admitting graph plans only when the recall model predicts Recall@10 ≥ 0.9 raises recall to 0.991 and 0.989 on the real data, at 1.47 and 1.01 ms. It mostly excludes ACORN, whose recall collapses at low local selectivity.

What limits HVO. Two factors account for most of the remaining gap to the cost model with true selectivities:

- *Planning time.* Excluding it, HVO's choices average 1.24, 0.74, 0.59 and 1.00 ms. The native planner decides in 90, 81, 45 and 89 μs per query (median; its choices agree with the Python implementation on 100% of test queries). That is significant when the best plan takes a few hundred microseconds.
- *Cost-model calibration.* For the best plan, the model's error is median $\kappa_{\text{cal}} = 1.19$ (P95 2.06) on ARXIV-384.

ACORN's cost could not be fitted by any smooth staircase: its adaptive stopping makes most queries cheap and a few very expensive, and the fitted variable term collapsed to zero. With that fit, the cost model sent rare-match queries to ACORN, and its mean on SYN-768 was worse than the global router's even with true selectivities. We therefore model ACORN with a monotone (isotonic) regression in the k-th-match horizon. Theorems 4.3 and 4.4 only need monotone costs, and for this plan Λ is computed numerically instead of by Lemma 4.2.

5.4 Inside PostgreSQL

Table 10 runs the same test queries as SQL against PostgreSQL with `pgvector`, and Figure 8 plots each configuration's recall against its latency.

Stock `pgvector` *and Exqutor are fast because they return incomplete answers.* With default settings, `pgvector`'s HNSW scan stops after $ef_{\text{search}} = 40$ candidates. For filtered queries it then returns fewer than k rows or rows outside the true top-k: mean recall 0.540 on ARXIV-384, with 56.8% of queries below 0.9. Exqutor behaves the same way (recall 0.538): as its design predicts, its estimator does not activate on single-table top-k queries.

`pgvector`*'s iterative scan improves recall but not enough.* With `strict_order`, recall rises to 0.846 and 0.904, with 28.1% and 16.8% of queries still below 0.9. The scan stops at `max_scan_tuples`=20,000 tuples, which anti-correlated queries exhaust.

HVO in PostgreSQL. The hook routes each query either to the pre-filter plan (filter first, exact distance sort) or to the iterative scan with a larger tuple budget. Compared with forcing that iterative scan for every query, it is both faster and more accurate on both datasets: 16.10 vs. 18.04 ms with recall 0.953 vs. 0.907 on ARXIV-384, and 4.45 vs. 5.69 ms with recall 0.953 vs. 0.931 on AMAZON-384. Its mean latency is below the oracle's (24.17 and 10.16 ms), whose recall target forces many slow pre-filter plans. Planning in the hook takes 126 and 119 μs (median). With the recall constraint, HVO reaches recall 0.984 and 0.988, close to the oracle's (0.986 and 0.991). In PostgreSQL that is expensive (mean 50.00 and 43.64 ms), because the pre-filter plan scans the heap and computes distances in the

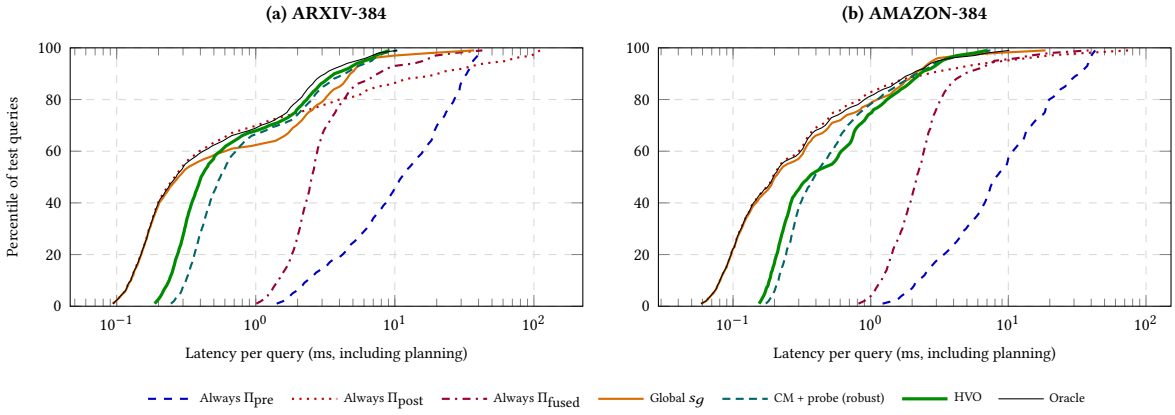


Figure 6: Distribution of per-query latency on the real test splits ($k = 10$; percentiles 1–99, logarithmic latency axis in ms, including planning time). Curves further left are faster. "Global s_g": best tuned global-selectivity router; "CM + probe": the calibrated cost model fed the HNSW probe, with minimax routing. The always-post curve has the fastest body and the slowest tail; HVO follows the faster plans in the body and cuts the tail.

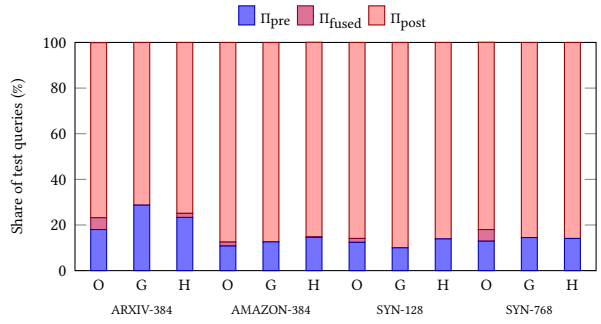


Figure 7: Plan choices at $k = 10$: share of test queries routed to each plan by the oracle (O), the best tuned global-selectivity router (G) and HVO (H). The global router is two-way on every dataset, so it never chooses Π_{fused}. HVO and the global router send similar shares to each plan; they differ in *which* queries they send (plan accuracy in Tables 8 and 9).

executor: a median of 139.62 ms against 11.20 ms for our columnar scan.

In PostgreSQL the choice is therefore a recall–latency trade-off, not a single speedup. The configurations most users would run are either fast and wrong (defaults) or still wrong on 17–28% of queries (iterative scan). HVO offers two operating points that dominate the forced iterative scan.

5.5 Effect of k

Table 13 and Figure 9 vary k. At $k = 100$, HVO's mean is 30% and 27% below the global router's on the real data, and its P99 regret is 2–10× lower on every dataset. At $k = 1$ the picture reverses on three of four datasets. Single-result queries are so cheap that planning time dominates, and both HPBS and the probe estimate poorly, because the horizon is a single match (h_1, P99 Q-errors in the tens to hundreds). HVO is then slower than the threshold router except on ARXIV-384 (0.83 vs. 1.38 ms). Routing pays off once queries cost more than planning. Figure 9 adds paired 95% intervals: the gain is clearly positive on ARXIV-384 at every k and on AMAZON-384 at $k = 100$, while the intervals for SYN-128 and SYN-768 at $k = 10$ and $k = 100$ and for AMAZON-384 at $k \leq 10$ include zero. At $k = 1$, HVO is detectably slower than the global router on SYN-768.

5.6 Transferability of the Calibrator

Table 14 trains the calibrator on one dataset and uses it on another. A calibrator trained only on SYN-128 brings the P99 Q-error on ARXIV-384 to 5.82, against 6.17 when trained on ARXIV-384 itself and 9.95 without a calibrator. The calibrator learns the sketch's systematic biases, which are largely dataset independent. Transfer into AMAZON-384 is the weakest: its query-anchored workload has the within-cell correlation the other datasets lack.

5.7 Effect of the Plan Implementations

Table 15 compares two generations of graph plans. The resumable post-filter is far faster than restarting a search with a doubled budget (P99 110.80 against several seconds on ARXIV-384), and ACORN-1 is faster than faiss's bitmap-filtered HNSW but loses recall at low local selectivity. HVO benefits from the better plans: its mean on ARXIV-384 falls from 1.84 ms with the first-generation plans to 1.41 ms.

5.8 Ablations and Scan Kernels

Scan kernels. Table 11 repeats routing with each pre-filter kernel. On ARXIV-384 the NEON FP32 kernel is 5.2× cheaper per vector than the scalar kernel, and INT8 `SDOT` with FP32 rerank is another 5.3× cheaper; the ratios are similar on the other datasets. Faster kernels widen the pre-filter region (Figure 3), and on every dataset both the oracle and HVO pick Π_{pre} more often as the kernel gets faster. The calibrated cost model follows the shift because only c_{scan} changes, so routing must be calibrated per kernel and machine.

Component ablations. Table 12 removes one component at a time:

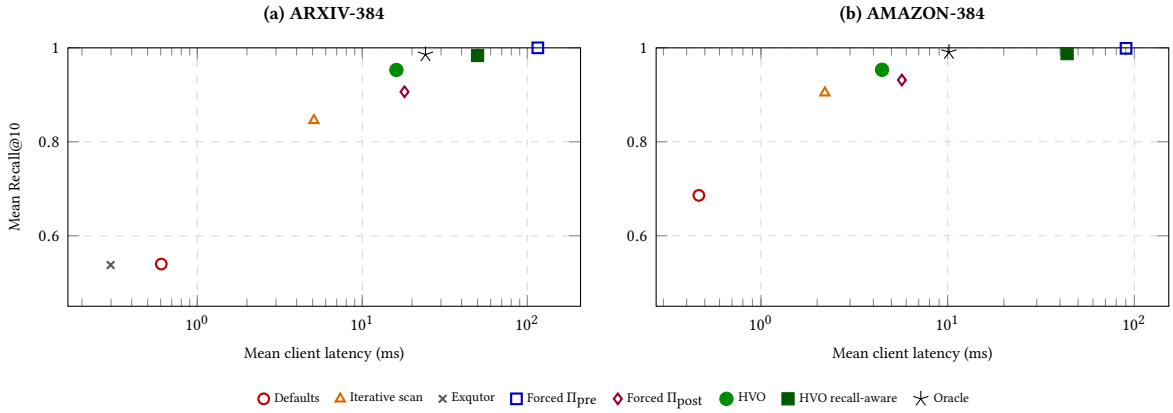


Figure 8: Recall–latency trade-off inside PostgreSQL 16 + `pgvector` ($k = 10$, test split; logarithmic latency axis). Up and to the left is better. Defaults and Exqutor are fast but incomplete; HVO lies above and to the left of the forced iterative post-filter on both datasets, and the recall-aware variant approaches the oracle's recall at a higher latency. Exqutor was run on ARXIV-384 only.

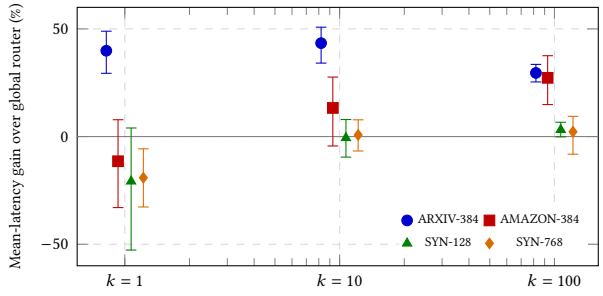


Figure 9: Relative mean-latency gain of HVO over the best tuned global-selectivity router, $1 - \overline{\text{lat}}_{\text{HVO}}/\overline{\text{lat}}_{\text{global}}$, with paired 95% bootstrap intervals over test queries (2000 resamples). Points are offset horizontally per dataset for legibility. Intervals that cross zero indicate no detectable difference.

Table 11: Scan kernel sensitivity. c_{scan}: fitted cost per scanned vector. s_g^* (%): pre-filter boundary at $s_{\text{loc}} = 0.3\%$ and 1% for two predicate terms (Theorem 3.2; 100 means pre-filtering wins for every s_g, 0 that it never does). #pre: queries for which the oracle / HVO choose Π_{pre}. Latencies in ms.

Data	Kernel	c_{scan} (ns)	s_g^*@0.3%	s_g^*@1%	#pre opt	#pre HVO	HVO mean	HVO P99	Oracle mean
ARXIV-384 (real)	Scalar FP32	240.3	1.5	0.2	168	181	1.96	19.22	2.52
ARXIV-384 (real)	NEON FP32	46.0	8.1	0.8	265	343	1.41	9.19	1.28
ARXIV-384 (real)	NEON INT8	8.7	41.6	2.9	343	421	1.09	5.16	0.89
AMAZON-384 (real)	Scalar FP32	248.5	1.2	0.0	108	142	1.32	12.14	1.72
AMAZON-384 (real)	NEON FP32	46.9	6.4	0.2	157	212	0.96	7.00	0.86
AMAZON-384 (real)	NEON INT8	8.7	33.0	0.0	190	244	0.83	3.91	0.60
SYN-128	Scalar FP32	64.4	1.0	0.0	150	187	0.80	3.86	0.83
SYN-128	NEON FP32	16.0	3.2	0.0	186	209	0.73	3.21	0.67
SYN-128	NEON INT8	3.3	12.0	0.0	207	217	0.71	3.07	0.60
SYN-768	Scalar FP32	564.4	0.3	0.0	128	134	1.48	11.39	2.99
SYN-768	NEON FP32	107.6	1.6	0.0	193	212	1.19	5.57	1.45
SYN-768	NEON INT8	25.2	7.0	0.0	286	277	1.06	3.77	1.07

- *Counted instead of measured costs.* Replacing measured costs by distance-evaluation counts makes sequential and random distance computations cost the same. HVO's mean rises from 1.41 to 2.25 ms on ARXIV-384, from 0.96 to 5.19 ms on AMAZON-384 and from 0.73 to 4.60 ms on SYN-128. Counting distances ignores that random accesses cost far more than sequential ones and that ACORN must first materialize the predicate map, so the agnostic model sends many queries to ACORN.
- *Calibrator.* Without it, HVO is slower on every dataset (1.55, 1.17, 0.83 and 1.50 ms), because the raw sketch's heavier tail forces a wider interval.
- *Point vs. minimax routing.* Under HPBS's actual error distribution the two differ by a few percent in mean and P99 latency (Tables 8 and 9). Minimax routing's value here is its guarantee (Theorem 4.4), not a measured gain.
- *Interval width* (Figure 10). No width is best on every dataset. Widths up to the 0.95-quantile of the training Q-error stay within about 15% of one another in mean latency, and point routing is the fastest in mean on ARXIV-384. The widest interval ($q = 0.99$) is the slowest in mean on all four datasets, because it routes too many queries to the exact scan. We keep $q = 0.9$, fixed before looking at test data, for its guarantee rather than for a measured gain.
- *Runtime guard.* The early-exit guard never fired with the resumable post-filter, so it contributes nothing; we keep it in the table for completeness.

5.9 Regret under Injected Estimation Error

To test Theorems 4.3 and 4.4, we replace the estimate by $\hat{s}_{\text{loc}} = s_{\text{loc}}\,\eta^{\pm 1}$ with a random sign (an adversarial, interval-endpoint error), keep $\hat{s}_g = s_g$, and route the SYN-128 test queries 20 times per value of η (Figure 11). Across 300000 routed queries the model regret never exceeded the bound of Lemma 4.2 and Theorems 4.3–4.4 (0 violations). Minimax routing had zero regret on every query whose best plan was Π_{pre}.

The bounds are loose. Already at $\eta = 1.1$ the bound is 5.30, because one extra doubling round of the post-filter, or a step of ACORN's isotonic cost curve, can be crossed by an arbitrarily small error; the observed P99 model regret is 1.00. The figure also shows

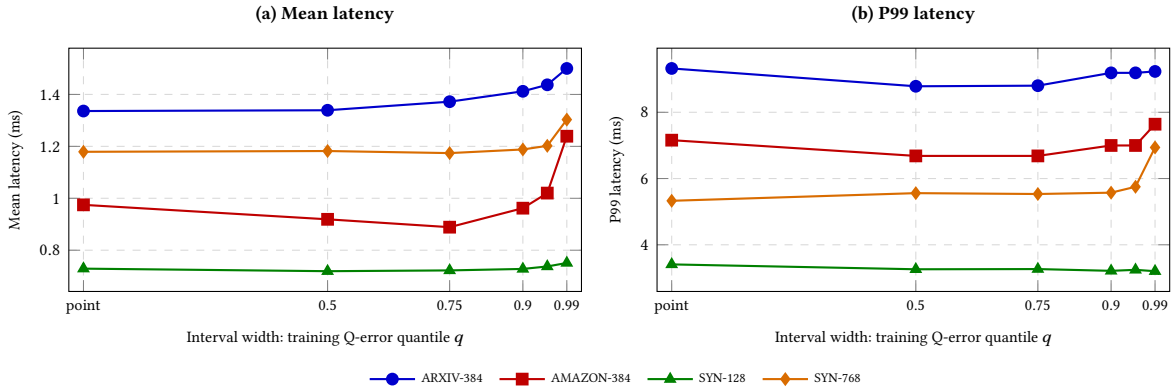


Figure 10: Effect of the uncertainty-interval width on HVO ($k = 10$, test split, latency including planning). "point": point routing ($\eta = 1$); otherwise η is the q-quantile of the calibrated sketch's Q-error on the training split. HVO's default is $q = 0.9$. No width is best on every dataset; the widest interval ($q = 0.99$) has the highest mean latency on all four.

Table 12: Ablations of HVO ($k = 10$): mean latency (ms), mean Recall@10, and P99 measured regret.

	ARXIV-384			AMAZON-384			SYN-128			SYN-768		
Variant	Mean	Rec.	P99 reg.	Mean	Rec.	P99 reg.	Mean	Rec.	P99 reg.	Mean	Rec.	P99 reg.
HVO (robust, L=4, calibrated)	1.41	0.988	3.21	0.96	0.988	6.54	0.73	0.981	3.27	1.19	0.967	2.79
point routing (no interval)	1.34	0.985	2.79	0.97	0.985	3.38	0.73	0.980	3.22	1.18	0.966	2.79
+ recall constraint	1.47	0.991	3.38	1.01	0.989	6.85	0.73	0.981	3.27	1.29	0.967	3.07
without calibrator	1.55	0.989	5.06	1.17	0.990	20.23	0.83	0.983	5.02	1.50	0.969	7.09
L=1 partition	1.31	0.988	3.03	0.83	0.989	6.05	0.65	0.981	2.04	1.07	0.967	1.77
counted cost (distance evaluations)	2.25	0.972	11.78	5.19	0.981	17.71	4.60	0.709	181.02	6.40	0.750	92.04
+ runtime early-exit guard	1.41	0.988	3.21	0.96	0.988	6.54	0.73	0.981	3.27	1.19	0.967	2.79
interval q=0.75	1.37	0.987	2.84	0.89	0.987	3.59	0.72	0.981	3.26	1.17	0.967	2.77
interval q=0.95	1.44	0.988	3.31	1.02	0.988	12.00	0.74	0.981	3.29	1.20	0.967	2.79
planning time excluded	1.24	0.988	3.06	0.74	0.988	5.93	0.59	0.981	1.69	1.00	0.967	1.52

Table 13: Varying k: mean latency (ms) of HVO, the best tuned global-selectivity router, the robust probe-based cost model, the cost model with true selectivities, and the oracle; mean Recall@k of HVO and the oracle; P99 measured regret of HVO and the global router; and P99 Q-error of HPBS and of the HNSW probe.

		Mean latency (ms)					Recall		P99 regret		P99 Q-error	
Data	k	HVO	Global	Probe	True sel.	Oracle	HVO	Oracle	HVO	Global	HPBS	Probe
ARXIV-384 (real)	1	0.83	1.38	1.09	0.59	0.67	0.973	1.000	6.02	22.11	36.59	174.98
	10	1.41	2.49	1.62	1.14	1.28	0.988	0.992	3.21	22.82	6.17	39.23
	100	2.88	4.09	3.43	2.63	2.67	0.993	0.994	1.78	19.05	2.63	13.76
AMAZON-384 (real)	1	0.72	0.65	0.71	0.43	0.64	0.975	0.996	7.04	11.71	116.77	42.67
	10	0.96	1.11	0.93	0.62	0.86	0.988	0.994	6.54	13.71	16.31	14.86
	100	1.76	2.42	1.94	1.45	1.64	0.991	0.993	2.60	8.09	4.60	16.87
SYN-128	1	0.39	0.32	0.45	0.22	0.29	0.979	1.000	5.56	1.85	23.09	42.67
	10	0.73	0.72	0.76	0.58	0.67	0.981	0.992	3.27	4.85	3.08	5.67
	100	2.14	2.21	2.58	1.97	1.97	0.992	0.994	2.08	5.08	1.78	5.52
SYN-768	1	0.68	0.57	0.81	0.44	0.85	0.953	1.000	5.47	7.85	25.61	31.44
	10	1.19	1.20	1.37	0.98	1.45	0.967	0.989	2.79	5.52	4.31	6.06
	100	4.13	4.22	4.79	3.67	4.95	0.985	0.990	2.04	4.94	1.90	6.67

the cost of conservatism. When every estimate sits at an endpoint of its interval, minimax routing is more conservative than point routing (P99 model regret 2.59 vs. 1.79 at $\eta = 2$).

6 Discussion

6.1 Limitations

Data and workloads. Two of the four datasets are real, but none of the predicate workloads is a real query log; no public log of filtered vector queries exists. The arXiv predicates are random, and the Amazon predicates are derived from the query product. HVO's advantage was larger on real data than on synthetic data, which

Table 14: Calibrator transfer: the calibrator and its interval width are trained on one dataset (rows) and used on another (columns); the cost model stays calibrated on the target. Each cell: P99 Q-error / HVO mean latency (ms).

Train ↓ / Test →	ARXIV-384	AMAZON-384	SYN-128	SYN-768
ARXIV-384	6.17 / 1.41	31.54 / 0.95	4.41 / 0.77	4.80 / 1.28
AMAZON-384	16.34 / 1.40	16.31 / 0.96	11.52 / 0.77	10.38 / 1.23
SYN-128	5.82 / 1.34	24.51 / 0.91	3.08 / 0.72	4.38 / 1.18
SYN-768	7.37 / 1.36	24.77 / 0.88	3.57 / 0.73	4.31 / 1.18
no calibrator	9.95 / 1.55	29.86 / 1.17	9.83 / 0.83	14.11 / 1.50

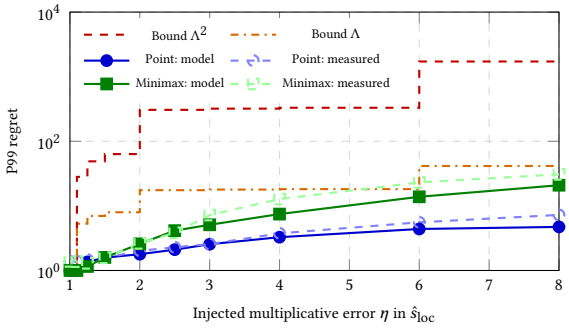


Figure 11: Regret on SYN-128 under injected estimation error ($\hat{s}_g = s_g$; 20 repetitions). Model regret uses the calibrated cost model at the true selectivities; measured regret uses the executed plans' latencies and includes the cost model's calibration error, which is why it exceeds 1 even at $\eta = 1$. Bounds are clipped at 10^4.

suggests that real correlation is stronger than our generator's. On the anchored Amazon workload, though, index probing estimated better than HPBS. Within-cell correlation is the sketch's blind spot (Proposition 4.1), and a workload that filters on attributes of the query itself is exactly what produces it.

One machine, one graph library. All results come from an Apple M4. We did not run on an x86 server, so the scan kernels are NEON and the AVX-512 behavior of the original design is untested. Absolute latencies and fitted constants must be recalibrated per machine; the structure of the cost model carries over. Graph-plan costs depend on implementation details: restarting a faiss search, the resumable scan, and ACORN have very different cost curves. One of them (ACORN) could only be modeled nonparametrically.

ACORN. We used the reference implementation, after fixing two heap overflows that crashed it beyond about 10^5 vectors on a hardened allocator. In our setting its recall at low local selectivity was poor, and $\gamma = 12$ did not help. A better-tuned ACORN would strengthen the fused plan, and with it the case for three-way routing, which our results barely exercise.

PostgreSQL integration. The hook steers PostgreSQL's planner through plan-type settings rather than adding new cost-based paths. Because `pgvector` has no in-traversal filtering, routing there is two-way. The pre-filter plan in PostgreSQL is far slower than our columnar scan, which makes the recall-aware variant expensive.

Small k and planning time. At $k = 1$, queries are cheap enough that 45–90 μs of planning outweighs better routing, and both HPBS and the probe estimate poorly. An optimizer should skip HVO when the cheapest plan is predicted to cost less than planning.

Statistical scope. The 95% intervals cover query sampling on one run per configuration; run-to-run timing variation was not replicated, and the intervals are not corrected for multiple comparisons. The mean gain on AMAZON-384 for the default configuration, and on both synthetic datasets, is within these intervals' uncertainty.

Not evaluated. We have not evaluated updates, `GROUP BY` top-k (which changes the post-filtering horizon from k matches to k distinct groups), multi-way joins, or skewed foreign keys in isolation.

6.2 When to Use HVO

The results suggest a few practical rules. (1) Route only when it can pay off: if the cheapest plan is predicted to cost less than planning itself (small k, very selective or very unselective filters), skip HVO and use the default plan. (2) Calibrate on the machine that will run the queries: the cost constants are fitted from forced runs of the training queries (1000 per dataset here), while the calibrator transfers reasonably across datasets (Section 5.6). (3) Where index probing is cheap and matches crowd around the query, as in AMAZON-384's anchored workload, a probe-based estimate is competitive; HPBS is most valuable when matches are rare near the query and when planning must not touch the index. (4) Inside PostgreSQL, decide first whether the application needs complete answers: the recall-aware variant returns nearly the oracle's recall but pays for PostgreSQL's slow pre-filter plan, while the latency-optimizing variant is faster and still more accurate than the iterative scan.

7 Conclusion

Choosing a plan for filtered vector search requires knowing how far the search must travel before it meets k matches. We defined that distance as the k-th-match local selectivity and showed that it is what global-selectivity routing misses. A compact sketch estimates it without touching the index and stays accurate on rare local matches, where index probing fails, and a minimax router with measured plan costs turns the estimate into a plan with provable regret bounds. Inside PostgreSQL, HVO is both faster and more accurate than `pgvector`'s iterative scan, on workloads where stock `pgvector` and Exqutor reach a mean recall of only about 0.540. In the benchmark it lowers mean latency on arXiv by 43% (95% interval 34–51%) relative to tuned global-selectivity routing and cuts tail latency several-fold on both real datasets; on Amazon and on synthetic data the default configuration's mean gain is not distinguishable from zero. The gains vanish at $k = 1$, routing works only if plan costs are measured rather than counted, and all results come from one machine. Future work should evaluate x86 servers and other graph libraries, maintain the sketch under updates, add cost-based paths to PostgreSQL instead of steering plan-type settings, and test the method on real query logs once they become available.

Table 15: First- versus second-generation graph plans ($k = 10$): mean and P99 latency (ms) and mean recall when the plan is used for every query, and HVO's mean latency with each plan set.

	Post: restart			Post: resumable			Fused: faiss bitmap			Fused: ACORN-1			HVO mean	
Data	Mean	P99	Rec.	Mean	P99	Rec.	Mean	P99	Rec.	Mean	P99	Rec.	Gen. 1	Gen. 2
ARXIV-384 (real)	691.43	15122.73	0.968	7.83	110.80	0.966	833.57	15509.94	0.978	5.40	42.59	0.930	1.84	1.41
SYN-128	33.74	548.34	0.978	1.28	13.41	0.976	117.87	2605.96	0.983	5.39	64.01	0.619	0.96	0.73
SYN-768	41.94	577.76	0.959	2.23	39.57	0.961	160.87	2702.76	0.974	18.67	76.82	0.591	2.04	1.19

Declarations and Reproducibility

Funding and conflicts of interest: This research received no external funding. The authors declare no conflicts of interest.

Code and data: Implementation, data generator, experiment drivers, raw results and the scripts that generate every number, table and figure in this paper are available at https://github.com/sanmish4ds/HyperVectorOptimizer.

Benchmark availability: ARXIV-384 and AMAZON-384 are published as a public Hugging Face dataset [32]. Each query comes with its filter, its global and k-th-match local selectivity, and its exact filtered top-k for $k \in \{1, 10, 100\}$; an evaluation script and reference baselines are included. ARXIV-384 includes the vectors and attributes (the source is CC0). The Amazon source states no license, so AMAZON-384 ships product identifiers only, with a script that rebuilds its attributes and embeddings from the original metadata and verifies them against published checksums.

Author contributions: S. Mishra: conceptualization, methodology, formal analysis, software, writing (original draft). **D. Chukkapalli:** validation, investigation, visualization, writing (review and editing).

References

[1] C. Wei *et al.*, "AnalyticDB-V: A hybrid analytical engine towards query fusion for structured and unstructured data," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3152–3165, 2020.

[2] Q. Zhang *et al.*, "VBase: Unifying online vector similarity search and relational queries via relaxed monotonicity," in *Proc. USENIX OSDI*, 2023, pp. 377–395.

[3] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "ACORN: Performant and predicate-agnostic search over vector embeddings and structured data," *Proc. ACM Manag. Data*, vol. 2, no. 3, 2024.

[4] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu, "Filtered approximate nearest neighbor search in vector databases: System design and performance analysis," arXiv:2602.11443, 2026.

[5] Y. Lin, K. Zhang, Z. He, Y. Jing, and X. S. Wang, "Survey of filtered approximate nearest neighbor search over the vector-scalar hybrid data," arXiv:2505.06501, 2025.

[6] P. Iff *et al.*, "Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors," arXiv:2507.21989, 2025.

[7] J. Shi, Y. Cai, and W. Zheng, "Filtered approximate nearest neighbor search: A unified benchmark and systematic experimental study," arXiv:2509.07789, 2025.

[8] A. Kane, "pgvector: Open-source vector similarity search for Postgres," 2024. [Online]. Available: https://github.com/pgvector/pgvector

[9] DuckDB, "Vector similarity search extension (`vss`)," 2024. [Online]. Available: https://duckdb.org/docs/extensions/vss

[10] J. Wang *et al.*, "Milvus: A purpose-built vector data management system," in *Proc. ACM SIGMOD*, 2021, pp. 2614–2627.

[11] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.

[12] M. Douze *et al.*, "The Faiss library," arXiv:2401.08281, 2024.

[13] S. Gollapudi *et al.*, "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in *Proc. ACM Web Conf.*, 2023, pp. 3406–3416.

[14] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, "Navigable proximity graph-driven native hybrid queries with structured and unstructured constraints," arXiv:2203.13601, 2022.

[15] W. Wu *et al.*, "HQANN: Efficient and robust similarity search for hybrid queries with structured and unstructured constraints," in *Proc. ACM CIKM*, 2022, pp. 4580–4584.

[16] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng, "SeRF: Segment graph for range-filtering approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 1, 2024.

[17] Z. Peng, M. Qiao, W. Zhou, F. Li, and D. Deng, "Dynamic range-filtering approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 18, no. 10, pp. 3256–3268, 2025.

[18] H. Kim, C. Lim, H. An, R. Sen, and K. Park, "Exqutor: Extended query optimizer for vector-augmented analytical queries," in *Proc. IEEE ICDE*, 2026. arXiv:2512.09695; code: https://github.com/BDAI-Research/Exqutor.

[19] P. G. Selinger *et al.*, "Access path selection in a relational database management system," in *Proc. ACM SIGMOD*, 1979, pp. 23–34.

[20] Y. Matias, J. S. Vitter, and M. Wang, "Wavelet-based histograms for selectivity estimation," in *Proc. ACM SIGMOD*, 1998, pp. 448–459.

[21] D. Gunopulos, G. Kollios, V. J. Tsotras, and C. Domeniconi, "Approximating multi-dimensional aggregate range queries over real attributes," in *Proc. ACM SIGMOD*, 2000, pp. 463–474.

[22] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, "HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm," in *Proc. AofA*, 2007, pp. 127–146.

[23] K. Beyer, P. J. Haas, B. Reinwald, Y. Sismanis, and R. Gemulla, "On synopses for distinct-value estimation under multiset operations," in *Proc. ACM SIGMOD*, 2007, pp. 199–210.

[24] A. Kipf *et al.*, "Learned cardinalities: Estimating correlated joins with deep learning," in *Proc. CIDR*, 2019.

[25] Z. Yang *et al.*, "NeuroCard: One cardinality estimator for all tables," *Proc. VLDB Endow.*, vol. 14, no. 1, pp. 61–73, 2020.

[26] B. Hilprecht, A. Schmidt, M. Kulessa, A. Molina, K. Kersting, and C. Binnig, "DeepDB: Learn from data, not from queries!" *Proc. VLDB Endow.*, vol. 13, no. 7, pp. 992–1005, 2020.

[27] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. ACM STOC*, 2002, pp. 380–388.

[28] R. Spring and A. Shrivastava, "A new unbiased and efficient class of LSH-based samplers and estimators for partition function computation in log-linear models," arXiv:1703.05160, 2017.

[29] H. Lan, S. Huang, Z. Bao, and R. Borovica-Gajic, "Cardinality estimation for similarity search on high-dimensional data objects: The impact of reference objects," *Proc. VLDB Endow.*, vol. 18, no. 3, pp. 544–556, 2024.

[30] "arxiv-abstracts-2021: metadata and abstracts of arXiv papers," Hugging Face dataset `gfissore/arxiv-abstracts-2021`, CC0 license, 2021. [Online]. Available: https://huggingface.co/datasets/gfissore/arxiv-abstracts-2021

[31] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, and J. McAuley, "Bridging language and items for retrieval and recommendation," arXiv:2403.03952, 2024.

[32] S. Mishra and D. Chukkapalli, "Correlated filtered vector search benchmark (ARXIV-384, AMAZON-384)," Hugging Face dataset `sanmish4/correlated-filtered-vector-search`, 2026. [Online]. Available: https://huggingface.co/datasets/sanmish4/correlated-filtered-vector-search

[33] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.